

Important Code Reference: server.mjs

A focused code/control-file explanation covering purpose, owner layer, APIs, variables, implementation signals, source excerpts, and debugging rules.

Item	Explanation
File	server.mjs
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	147,116 bytes
Extension	.mjs
MIME type	text/plain
Text lines	3,632

Why this file exists

- Local backend server used by the builder for drafts, parsing, publishing, compiler execution, authentication checks, and file operations.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Imports, requires, and external dependencies

Item	Explanation
Line 1	import { createServer } from "node:http";
Line 2	import { execFile, spawn } from "node:child_process";
Line 3	import { createHash } from "node:crypto";
Line 4	import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from "node:fs/promises";
Line 5	import os from "node:os";
Line 6	import path from "node:path";
Line 7	import { promisify } from "node:util";
Line 8	import { fileURLToPath } from "node:url";

API endpoints mentioned or called

Item	Explanation
/api/catalog	Line 3343. Loads project/catalog data for the builder.
/api/templates	Line 3353. Loads appearance template definitions.
/api/publish-target	Line 3358. Reads or writes the Git publishing target and authorization state.
/api/system-check	Line 3367. Checks local tool/system readiness.
/api/app-update	Line 3372. Checks or starts builder app update behavior.
/api/security-report	Line 3377. Returns security/download/auth reporting for the builder.
/api/code/tools	Line 3386. Checks compiler/runtime availability.

Item	Explanation
/api/portfolio-ai	Line 3402. Handles AI assistant questions.
/api/code/beautify	Line 3407. Formats source code for cleaner display and editing.
/api/code/save	Line 3422. Saves source code into the local compile workspace.
/api/code/compile	Line 3433. Compiles or runs a source file and returns terminal output.
/api/code/install-tools	Line 3444. Attempts to install missing compiler tools.
/api/publish-target	Line 3455. Reads or writes the Git publishing target and authorization state.
/api/install-git	Line 3465. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/app-update/install	Line 3479. Checks or starts builder app update behavior.
/api/github-authenticate	Line 3493. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/sync-from-publish-target	Line 3510. Reads or writes the Git publishing target and authorization state.
/api/save-draft	Line 3525. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/apply-catalog	Line 3525. Loads project/catalog data for the builder.
/api/apply-catalog	Line 3534. Loads project/catalog data for the builder.
/api/upload	Line 3577. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.

Important implementation signals

Item	Explanation
Compiler or simulator at line 1	Part of the Compile Code workspace or language/tool detection. Evidence: import { createServer } from "node:http";
Process execution at line 2	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: import { execFile, spawn } from "node:child_process";
Compiler or simulator at line 3	Part of the Compile Code workspace or language/tool detection. Evidence: import { createHash } from "node:crypto";
File read at line 4	Reads local builder state, project files, website assets, or configuration. Evidence: import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from "node:fs/promises";
Compiler or simulator at line 5	Part of the Compile Code workspace or language/tool detection. Evidence: import os from "node:os";
Compiler or simulator at line 6	Part of the Compile Code workspace or language/tool detection. Evidence: import path from "node:path";
Compiler or simulator at line 7	Part of the Compile Code workspace or language/tool detection. Evidence: import { promisify } from "node:util";
Compiler or simulator at line 8	Part of the Compile Code workspace or language/tool detection. Evidence: import { fileURLToPath } from "node:url";
Process execution at line 15	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: const execFileAsync = promisify(execFile);
File read at line 16	Reads local builder state, project files, website assets, or configuration. Evidence: const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
Browser DOM at line 34	Builder or website interface behavior in the browser/Electron renderer. Evidence: ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
Security or auth at line 40	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
Security or auth at line 51	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers",
Security or auth at line 57	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
Security or auth at line 58	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
Security or auth at line 59	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
Security or auth at line 60	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthExtendedThreshold = 3;
Security or auth at line 63	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthorizationHelp = [

Item	Explanation
Security or auth at line 65	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Open Publishing target, enter the repository URL, then click Authenticate target.",
Git command at line 66	Repository state, publishing, branch checks, or release automation. Evidence: "A GitHub/Git Credential Manager browser sign-in may open. Sign in with an account that has write access to the selected Pages repository.",
Security or auth at line 67	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Authenticate target only verifies write access and remembers it for the matching repository and branch.",
Security or auth at line 68	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Load from target only imports compatible website files from the authenticated repository into the local builder workspace.",
Git command at line 72	Repository state, publishing, branch checks, or release automation. Evidence: process.env.GIT_EXE,
Git command at line 73	Repository state, publishing, branch checks, or release automation. Evidence: "git",
Git command at line 74	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\cmd\\git.exe",
Git command at line 75	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\bin\\git.exe",
Git command at line 76	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
Git command at line 77	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\bin\\git.exe",
Git command at line 78	Repository state, publishing, branch checks, or release automation. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "cmd", "git.exe") : "",
Git command at line 79	Repository state, publishing, branch checks, or release automation. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "bin", "git.exe") : ""
Compiler or simulator at line 87	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["gcc"],
Compiler or simulator at line 97	Part of the Compile Code workspace or language/tool detection. Evidence: verilog: {
Compiler or simulator at line 100	Part of the Compile Code workspace or language/tool detection. Evidence: label: "Verilog",
Compiler or simulator at line 101	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["iverilog", "vvp"],
Compiler or simulator at line 102	Part of the Compile Code workspace or language/tool detection. Evidence: winget: ["Icarus.Verilog"]
Compiler or simulator at line 104	Part of the Compile Code workspace or language/tool detection. Evidence: systemverilog: {
Compiler or simulator at line 107	Part of the Compile Code workspace or language/tool detection. Evidence: label: "SystemVerilog",
Compiler or simulator at line 108	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["iverilog", "vvp"],
Compiler or simulator at line 109	Part of the Compile Code workspace or language/tool detection. Evidence: winget: ["Icarus.Verilog"]
Compiler or simulator at line 118	Part of the Compile Code workspace or language/tool detection. Evidence: java: {
Compiler or simulator at line 119	Part of the Compile Code workspace or language/tool detection. Evidence: defaultFile: "Main.java",
Compiler or simulator at line 120	Part of the Compile Code workspace or language/tool detection. Evidence: extensions: [".java"],
Compiler or simulator at line 121	Part of the Compile Code workspace or language/tool detection. Evidence: label: "Java",
Compiler or simulator at line 122	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["javac", "java"],
Compiler or simulator at line 129	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["node"],
Compiler or simulator at line 132	Part of the Compile Code workspace or language/tool detection. Evidence: python: {

Item	Explanation
Compiler or simulator at line 135	Part of the Compile Code workspace or language/tool detection. Evidence: label: "Python",
Compiler or simulator at line 136	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["python"],
Compiler or simulator at line 149	Part of the Compile Code workspace or language/tool detection. Evidence: gcc: [
Compiler or simulator at line 150	Part of the Compile Code workspace or language/tool detection. Evidence: "gcc",
Compiler or simulator at line 151	Part of the Compile Code workspace or language/tool detection. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages", "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "gcc.exe") : ""
Installer at line 156	Windows setup, update, shortcut, or installation behavior. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages", "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "g++.exe") : ""
Compiler or simulator at line 161	Part of the Compile Code workspace or language/tool detection. Evidence: javac: [
Compiler or simulator at line 162	Part of the Compile Code workspace or language/tool detection. Evidence: "javac",
Compiler or simulator at line 163	Part of the Compile Code workspace or language/tool detection. Evidence: "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\javac.exe"
Compiler or simulator at line 165	Part of the Compile Code workspace or language/tool detection. Evidence: java: [
Compiler or simulator at line 166	Part of the Compile Code workspace or language/tool detection. Evidence: "java",
Compiler or simulator at line 167	Part of the Compile Code workspace or language/tool detection. Evidence: "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\java.exe"
Compiler or simulator at line 169	Part of the Compile Code workspace or language/tool detection. Evidence: node: [
Compiler or simulator at line 170	Part of the Compile Code workspace or language/tool detection. Evidence: /(?:^[\\V])node(?:\\.exe)?\$/i.test(process.execPath "") ? process.execPath : "",
Compiler or simulator at line 171	Part of the Compile Code workspace or language/tool detection. Evidence: "node",
Compiler or simulator at line 172	Part of the Compile Code workspace or language/tool detection. Evidence: "C:\\Program Files\\nodejs\\node.exe",
Compiler or simulator at line 173	Part of the Compile Code workspace or language/tool detection. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "nodejs", "node.exe") : ""
Compiler or simulator at line 175	Part of the Compile Code workspace or language/tool detection. Evidence: python: [process.env.PYTHON, "python", "py"],
Compiler or simulator at line 176	Part of the Compile Code workspace or language/tool detection. Evidence: iverilog: ["iverilog", "C:\\iverilog\\bin\\iverilog.exe"],
Compiler or simulator at line 177	Part of the Compile Code workspace or language/tool detection. Evidence: vvp: ["vvp", "C:\\iverilog\\bin\\vvp.exe"],
Installer at line 180	Windows setup, update, shortcut, or installation behavior. Evidence: "C:\\Program Files\\ADI\\LTspice\\LTspice.exe",
Installer at line 181	Windows setup, update, shortcut, or installation behavior. Evidence: "C:\\Program Files\\LTC\\LTspiceXVII\\XVIIx64.exe",
Installer at line 182	Windows setup, update, shortcut, or installation behavior. Evidence: "C:\\Program Files\\Analog Devices\\LTspice\\LTspice.exe",
Installer at line 183	Windows setup, update, shortcut, or installation behavior. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "ADI", "LTspice", "LTspice.exe") : ""
Local storage at line 186	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: const compileToolCache = new Map();
Local storage at line 187	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: const compileToolVersionCache = new Map();
Cloudflare or AI at line 192	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Answer the visitor's question first in a concise ChatGPT-like format, then add portfolio links or context only when they help.",

Item	Explanation
Security or auth at line 210	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Do not invent portfolio projects, credentials, employers, files, or test results that are not in the context.",
Security or auth at line 218	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function securityHeaders(extra = {}) {
Security or auth at line 222	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",
Security or auth at line 230	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ...securityHeaders(),
Local storage at line 232	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: "Cache-Control": "no-store, no-cache, must-revalidate"
Compiler or simulator at line 263	Part of the Compile Code workspace or language/tool detection. Evidence: systemverilog: "systemverilog",
Compiler or simulator at line 264	Part of the Compile Code workspace or language/tool detection. Evidence: sv: "systemverilog",

Important constants and variables

Item	Explanation
root	Line 10: const root = path.dirname(fileURLToPath(import.meta.url));
portfolioRoot	Line 11: const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE root);
compileRoot	Line 12: const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE path.join(path.dirname(root), "compile-code"));
port	Line 13: const port = Number(process.env.PORT 8080);
host	Line 14: const host = process.env.HOST "0.0.0.0";
execFileAsync	Line 15: const execFileAsync = promisify(execFile);
packageJson	Line 16: const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
types	Line 18: const types = {
draftPath	Line 38: const draftPath = path.join(root, "projects.local.json");
catalogPath	Line 39: const catalogPath = path.join(root, "projects.json");
publishAuthCachePath	Line 40: const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
publishPaths	Line 41: const publishPaths = [
publishAuthCacheTtlMs	Line 57: const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
publishAuthExtendedTtlMs	Line 58: const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
publishAuthHistoryWindowMs	Line 59: const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
publishAuthExtendedThreshold	Line 60: const publishAuthExtendedThreshold = 3;
defaultSiteRepository	Line 61: const defaultSiteRepository = process.env.OMB_BUILDER_REPOSITORY "";
blockedAppUpdateVersions	Line 62: const blockedAppUpdateVersions = new Set(["0.2.16"]);
publishAuthorizationHelp	Line 63: const publishAuthorizationHelp = [
gitCandidates	Line 71: const gitCandidates = [
compileLanguageProfiles	Line 82: const compileLanguageProfiles = {
compileToolCandidates	Line 148: const compileToolCandidates = {
compileToolCache	Line 186: const compileToolCache = new Map();
compileToolVersionCache	Line 187: const compileToolVersionCache = new Map();
portfolioAiInstructions	Line 189: const portfolioAiInstructions = [
clean	Line 257: const clean = String(value "").trim().toLowerCase().replace(/[_\s-]+/g, "");
aliases	Line 258: const aliases = {

Item	Explanation
text	Line 283: const text = String(source "");
text	Line 289: const text = String(source "");
ext	Line 295: const ext = path.extname(String(fileName "").toLowerCase());
byName	Line 305: const byName = languageFromFileName(fileName, code);
source	Line 307: const source = String(code "");
profile	Line 322: const profile = compileLanguageProfiles[normalizeCodeLanguage(language)] compileLanguageProfiles.javascript;
fallback	Line 323: const fallback = profile.defaultFile;
parsed	Line 324: const parsed = path.parse(String(value fallback));
safeName	Line 325: const safeName = safeSegment(parsed.name, path.parse(fallback).name);
ext	Line 326: let ext = String(parsed.ext path.extname(fallback)).toLowerCase();
depth	Line 332: let depth = 0;
line	Line 337: const line = rawLine.trim();
formatted	Line 340: const formatted = `\${" ".repeat(depth)}\${line}`;
opens	Line 341: const opens = (line.match(/[{}()]/g) []).length;
closes	Line 342: const closes = (line.match(/[\[\]\]/g) []).length;
normalized	Line 351: const normalized = String(code "").replace(/\r\n?/g, "\n").replace(/\t/g, " ");
lang	Line 352: const lang = normalizeCodeLanguage(language);
entries	Line 376: let entries = [];
fullPath	Line 383: const fullPath = path.join(folder, entry.name);
found	Line 388: const found = await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1);
candidates	Line 396: const candidates = (compileToolCandidates[toolName] [toolName]).filter(Boolean);
command	Line 406: const command = process.platform === "win32" ? "where" : "command";
args	Line 407: const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
result	Line 408: const result = await execFileAsync(command, args, { timeout: 5000, windowsHide: true });
found	Line 409: const found = String(result.stdout "").split(/\r?\n/).map((line) => line.trim()).find(Boolean);
found	Line 420: const found = await findExecutableUnder(
found	Line 430: const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`\${toolName}.exe`], 5);
elapsedSeconds	Line 444: const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
elapsed	Line 445: const elapsed = elapsedSeconds ? `in \${elapsedSeconds}s` : "";
status	Line 446: const status = result.timedOut
output	Line 449: const output = [result.stdout, result.stderr].map((part) => String(part "").trimEnd()).filter(Boolean).join("\n");
clean	Line 454: const clean = String(value "");
output	Line 464: let output = String(text "");
from	Line 466: const from = replacement?.from "";
to	Line 467: const to = replacement?.to "";
isCpp	Line 481: const isCpp = language === "cpp";
ext	Line 482: const ext = path.extname(String(fileName "").toLowerCase());
header	Line 483: const header = [".h", ".hpp", ".hh", ".hxx"].includes(ext);
parsed	Line 504: const parsed = path.parse(String(fileName "program"));
result	Line 511: const result = await runProcess(toolPath, ["--version"], { timeoutMs: 7000 });

Item	Explanation
version	Line 512: const version = [result.stdout, result.stderr]
elapsedSeconds	Line 524: const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
status	Line 525: const status = result.timedOut
output	Line 528: const output = [result.stdout, result.stderr].map((part) => String(part "").trimEnd()).filter(Boolean).join("\n");
raw	Line 533: const raw = [result.stdout, result.stderr].map((part) => String(part "").trimEnd()).filter(Boolean).join("\n");
lines	Line 534: const lines = raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean);
finishLines	Line 535: const finishLines = [];
signalLines	Line 536: const signalLines = [];
elapsedSeconds	Line 544: const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
status	Line 545: const status = result.timedOut
body	Line 548: const body = signalLines.length
suffix	Line 555: const suffix = signalLines.length && finishLines.length ? ` \n\${finishLines.join("\n")}` : "";
timeoutMs	Line 560: const timeoutMs = options.timeoutMs 20000;

JSON/YAML-style keys detected

Item	Explanation
.css	Line 19: ".css": "text/css",
.csv	Line 20: ".csv": "text/csv",
.html	Line 21: ".html": "text/html",
.ico	Line 22: ".ico": "image/x-icon",
.js	Line 23: ".js": "application/javascript",
.json	Line 24: ".json": "application/json",
.md	Line 25: ".md": "text/markdown",
.pdf	Line 26: ".pdf": "application/pdf",
.png	Line 27: ".png": "image/png",
.jpg	Line 28: ".jpg": "image/jpeg",
.jpeg	Line 29: ".jpeg": "image/jpeg",
.svg	Line 30: ".svg": "image/svg+xml",
.txt	Line 31: ".txt": "text/plain",
.webp	Line 32: ".webp": "image/webp",
.xml	Line 33: ".xml": "application/xml",
.xlsx	Line 34: ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
.zip	Line 35: ".zip": "application/zip"
g++	Line 153: "g++": [
clang++	Line 159: "clang++": ["clang++"],
X-Content-Type-Options	Line 220: "X-Content-Type-Options": "nosniff",
Referrer-Policy	Line 221: "Referrer-Policy": "strict-origin-when-cross-origin",
Permissions-Policy	Line 222: "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",
Cross-Origin-Opener-Policy	Line 223: "Cross-Origin-Opener-Policy": "same-origin",
Content-Type	Line 231: "Content-Type": "application/json",
Cache-Control	Line 232: "Cache-Control": "no-store, no-cache, must-revalidate"

Item	Explanation
c++	Line 262: "c++": "cpp",
Accept	Line 1386: "Accept": accept,
User-Agent	Line 1387: "User-Agent": "OMB-Portfolio-Builder-AI"
Authorization	Line 3314: "Authorization": `Bearer \${apiKey}`,
Content-Type	Line 3315: "Content-Type": "application/json"
Content-Type	Line 3621: "Content-Type": types[path.extname(filePath)] "application/octet-stream",
Cache-Control	Line 3622: "Cache-Control": "no-store, no-cache, must-revalidate"

Function-by-function detail

securityHeaders (server.mjs, lines 218-226)

securityHeaders part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 218-226 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 219: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

218: function securityHeaders(extra = {}) {
219:   return {
220:     "X-Content-Type-Options": "nosniff",
221:     "Referrer-Policy": "strict-origin-when-cross-origin",
222:     "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",
223:     "Cross-Origin-Opener-Policy": "same-origin",
224:     ...extra
225:   };
226: }
```

sendJson (server.mjs, lines 228-235)

sendJson part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 228-235 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references writeHead, securityHeaders, end, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

228: function sendJson(response, status, data) {
229:   response.writeHead(status, {
230:     ...securityHeaders(),
231:     "Content-Type": "application/json",
232:     "Cache-Control": "no-store, no-cache, must-revalidate"
233:   });
234:   response.end(JSON.stringify(data, null, 2));
235: }
```

clampText (server.mjs, lines 237-239)

clampText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 237-239 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 238: return String(value || "").slice(0, maxLength);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
237: function clampText(value, maxLength = 12000) {
238:   return String(value || "").slice(0, maxLength);
239: }
```

stripHtmlToText (server.mjs, lines 241-254)

stripHtmlToText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 241-254 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 242: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
241: function stripHtmlToText(value = "") {
242:   return String(value || "")
243:     .replace(/<script[\s\S]*?<\script>/gi, " ")
244:     .replace(/<style[\s\S]*?<\style>/gi, " ")
245:     .replace(/<[>]+>/g, " ")
246:     .replace(/&nbsp;/gi, " ")
247:     .replace(/&amp;/gi, "&")
248:     .replace(/&lt;/gi, "<")
249:     .replace(/&gt;/gi, ">")
250:     .replace(/&quot;/gi, "'")
251:     .replace(/&#39;/g, "'")
252:     .replace(/\\s+/g, " ")
253:     .trim();
254: }
```

normalizeCodeLanguage (server.mjs, lines 256-280)

normalizeCodeLanguage validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 256-280 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 279: return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");.

Important local values include clean at line 257; aliases at line 258. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
256: function normalizeCodeLanguage(value = "") {
257:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
258:   const aliases = {
259:     c: "c",
260:     cpp: "cpp",
261:     cplusplus: "cpp",
262:     "c++": "cpp",
263:     systemverilog: "systemverilog",
264:     sv: "systemverilog",
265:     verilog: "verilog",
266:     v: "verilog",
```

```

267:     ltspice: "ltspice",
268:     spice: "ltspice",
269:     cir: "ltspice",
270:     java: "java",
271:     javascript: "javascript",
272:     js: "javascript",
273:     node: "javascript",
274:     python: "python",
275:     py: "python",
276:     html: "html",
277:     htm: "html"
278: };
279: return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");
280: }

```

sourceLooksCpp (server.mjs, lines 282-286)

sourceLooksCpp part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 282-286 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 284: return
 /#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text) ||.

Important local values include text at line 283. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

282: function sourceLooksCpp(source = "") {
283:   const text = String(source || "");
284:   return
  /#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text)
  ||
  285:     /\b(std::|cout|cin|cerr|namespace|template\s*<|class\s+\w+|new\s+\w+|delete\s+|constexpr|nullptr|using
\s+namespace)\b/.test(text);
286: }

```

sourceLooksC (server.mjs, lines 288-292)

sourceLooksC part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 288-292 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 290: return
 /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||.

Important local values include text at line 289. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

288: function sourceLooksC(source = "") {
289:   const text = String(source || "");
290:   return /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||
291:     /\b(printf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text);
292: }

```

languageFromFileName (server.mjs, lines 294-302)

languageFromFileName part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 294-302 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `extname`, `toLowerCase`, `sourceLooksCpp`, `sourceLooksC`, `entries`, `find`, and `includes`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 297: `if (sourceLooksCpp(code)) return "cpp";`; line 298: `if (sourceLooksC(code)) return "c";`; line 299: `return "c";`; line 301: `return Object.entries(compileLanguageProfiles).find(([, profile]) => profile.extensions.includes(ext))?.[0] || ""`;

Important local values include `ext` at line 295. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
294: function languageFromFileName(fileName = "", code = "") {
295:   const ext = path.extname(String(fileName || "").toLowerCase());
296:   if (ext === ".h") {
297:     if (sourceLooksCpp(code)) return "cpp";
298:     if (sourceLooksC(code)) return "c";
299:     return "c";
300:   }
301:   return Object.entries(compileLanguageProfiles).find(([, profile]) =>
profile.extensions.includes(ext))?.[0] || "";
302: }
```

detectCodeLanguageFromSource (server.mjs, lines 304-319)

`detectCodeLanguageFromSource` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 304-319 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `languageFromFileName`, `b`, `sourceLooksCpp`, and `sourceLooksC`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 306: `if (byName) return byName;`; line 308: `if (/<V?[a-z][\s\S]*?>/i.test(source) || /<!doctype\s+html/i.test(source)) return "html";`; line 309: `if (/b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+lw+)\b/i.test(source)) return "systemverilog";`; line 310: `if (/b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/i.test(source)) return "verilog";`. There are 6 additional return statements in the function body.

Important local values include `byName` at line 305; `source` at line 307. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
304: function detectCodeLanguageFromSource(code = "", fileName = "") {
305:   const byName = languageFromFileName(fileName, code);
306:   if (byName) return byName;
307:   const source = String(code || "");
308:   if (/<V?[a-z][\s\S]*?>/i.test(source) || /<!doctype\s+html/i.test(source)) return "html";
309:   if (/b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|
class\s+lw+)\b/i.test(source)) return "systemverilog";
310:   if (/b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/i.test(source)) return
"verilog";
311:   if (/^s*(?!(tran|ac|dc|op|model|subckt|ends|param)\b/im.test(source) ||
/bv\w+\s+\w+\s+\w+\s+(?:DC|SIN|PULSE)?/i.test(source)) return "ltspice";
312:   if (/b(def|elif|from\s+\w+\s+import|self|None|True|False)\b/i.test(source)) return "python";
313:   if (/b(public\s+class|static\s+void\s+main|System\.out)\b/i.test(source)) return "java";
314:   if (sourceLooksCpp(source) || sourceLooksC(source) ||
/b(#include|main\s*(|puts\s*(|fprintf|sizeof\s*(|)\b/i.test(source)) {
315:     return sourceLooksCpp(source) ? "cpp" : "c";
316:   }
317:   if (/b(const|let|var|function|=>|console\.log|document\.|window\.)\b/i.test(source)) return
"javascript";
318:   return "javascript";
319: }
```

safeCodeFileName (server.mjs, lines 321-329)

`safeCodeFileName` validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 321-329 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizeCodeLanguage`, `parse`, `safeSegment`, `extname`, `toLowerCase`, and `includes`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 328: `return `${safeName}${ext}``;

Important local values include profile at line 322; fallback at line 323; parsed at line 324; safeName at line 325; ext at line 326. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
321: function safeCodeFileName(value = "", language = "javascript") {
322:   const profile = compileLanguageProfiles[normalizeCodeLanguage(language)] ||
compileLanguageProfiles.javascript;
323:   const fallback = profile.defaultFile;
324:   const parsed = path.parse(String(value || fallback));
325:   const safeName = safeSegment(parsed.name, path.parse(fallback).name);
326:   let ext = String(parsed.ext || path.extname(fallback)).toLowerCase();
327:   if (!profile.extensions.includes(ext)) ext = path.extname(fallback);
328:   return `${safeName}${ext}`;
329: }
```

indentBraceCode (server.mjs, lines 331-345)

indentBraceCode part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 331-345 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, split, map, trim, max, repeat, and match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 333: return String(code || ""); line 338: if (!line) return ""; line 344: return formatted;.

Important local values include depth at line 332; line at line 337; formatted at line 340; opens at line 341; closes at line 342. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
331: function indentBraceCode(code = "") {
332:   let depth = 0;
333:   return String(code || "")
334:     .replace(/\r\n?/g, "\n")
335:     .split("\n")
336:     .map((rawLine) => {
337:       const line = rawLine.trim();
338:       if (!line) return "";
339:       if (/^[{}]\]/.test(line)) depth = Math.max(0, depth - 1);
340:       const formatted = `${" ".repeat(depth)}${line}`;
341:       const opens = (line.match(/[{}(]/g) || []).length;
342:       const closes = (line.match(/[})\)]/g) || []).length;
343:       depth = Math.max(0, depth + opens - closes);
344:       return formatted;
345:     })
}
```

beautifyCode (server.mjs, lines 350-372)

beautifyCode part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 350-372 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, normalizeCodeLanguage, includes, indentBraceCode, split, map, trim, join, and trimEnd. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 354: return indentBraceCode(normalized); line 359: return normalized; line 366: return normalized.

Important local values include normalized at line 351; lang at line 352. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
350: function beautifyCode(code = "", language = "javascript") {
351:   const normalized = String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ");
352:   const lang = normalizeCodeLanguage(language);
353:   if (["c", "cpp", "java", "javascript", "verilog", "systemverilog"].includes(lang)) {
354:     return indentBraceCode(normalized)
355:       .replace(/[ \t]+$/gm, "")
356:       .replace(/\n{4,}/g, "\n\n\n");
357:   }
358:   if (lang === "html") {
```

```

359:     return normalized
360:     .replace(/>\s+</g, ">\n<")
361:     .split("\n")
362:     .map((line) => line.trim())
363:     .join("\n")
364:     .trim();
365:   }
366:   return normalized
367:   .split("\n")
368:   .map((line) => line.trimEnd())
369:   .join("\n")
370:   .replace(/\n{4,}/g, "\n\n\n")
371:   .trimEnd();
372: }

```

findExecutableUnder (server.mjs, lines 374-392)

findExecutableUnder part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 374-392 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pathExists, readdir, join, isFile, some, toLowerCase, and isDirectory. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 375: if (!folder || maxDepth < 0 || !(await pathExists(folder))) return "";; line 380: return "";; line 384: if (entry.isFile() && names.some((name) => entry.name.toLowerCase() === name.toLowerCase())) return fullPath;; line 389: if (found) return found;. There are 1 additional return statements in the function body.

Important local values include entries at line 376; fullPath at line 383; found at line 388. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

374: async function findExecutableUnder(folder, names = [], maxDepth = 5) {
375:   if (!folder || maxDepth < 0 || !(await pathExists(folder))) return "";
376:   let entries = [];
377:   try {
378:     entries = await readdir(folder, { withFileTypes: true });
379:   } catch {
380:     return "";
381:   }
382:   for (const entry of entries) {
383:     const fullPath = path.join(folder, entry.name);
384:     if (entry.isFile() && names.some((name) => entry.name.toLowerCase() === name.toLowerCase())) return
fullPath;
385:   }
386:   for (const entry of entries) {
387:     if (!entry.isDirectory()) continue;
388:     const found = await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1);
389:     if (found) return found;
390:   }
391:   return "";
392: }

```

findTool (server.mjs, lines 394-437)

This function locates external tools such as compilers, runtimes, Git, or simulators. It checks known paths and command availability, then caches the result so repeated compile/run actions do not keep searching the machine.

Compile Code depends on tools that may or may not be installed. The builder needs fast, clear tool detection so it can explain missing tools and avoid lag every time the user presses Compile or Run.

It calls or references has, get, filter, isAbsolute, pathExists, set, execFileAsync, split, map, trim, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 395: if (compileToolCache.has(toolName)) return compileToolCache.get(toolName);; line 401: return candidate;; line 412: return found;; line 426: return found || "";. There are 2 additional return statements in the function body.

Important local values include candidates at line 396; command at line 406; args at line 407; result at line 408; found at line 409; found at line 420; found at line 430. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

394: async function findTool(toolName) {
395:   if (compileToolCache.has(toolName)) return compileToolCache.get(toolName);
396:   const candidates = (compileToolCandidates[toolName] || [toolName]).filter(Boolean);
397:   for (const candidate of candidates) {

```

```

398:     if (path.isAbsolute(candidate)) {
399:       if (await pathExists(candidate)) {
400:         compileToolCache.set(toolName, candidate);
401:         return candidate;
402:       }
403:       continue;
404:     }
405:     try {
406:       const command = process.platform === "win32" ? "where" : "command";
407:       const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
408:       const result = await execFileAsync(command, args, { timeout: 5000, windowsHide: true });
409:       const found = String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean);
410:       if (found) {
411:         compileToolCache.set(toolName, found);
412:         return found;
413:       }
414:     } catch {
415:       // Try the next candidate.
416:     }
417:   }
418: }
419: if ([ "gcc", "g++" ].includes(toolName) && process.env.LOCALAPPDATA) {
420:   const found = await findExecutableUnder(
421:     path.join(process.env.LOCALAPPDATA, "Microsoft", "winGet", "Packages"),
422:     [ toolName === "gcc" ? "gcc.exe" : "g++.exe" ],
423:     7
424:   );
425:   compileToolCache.set(toolName, found || "");
426:   return found || "";
427: }
428:
429: if ([ "javac", "java" ].includes(toolName)) {
430:   const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [ `${toolName}.exe` ],
5);
431:   compileToolCache.set(toolName, found || "");
432:   return found || "";
433: }
434:
435: compileToolCache.set(toolName, "");
436: return "";
437: }

```

terminalLine (server.mjs, lines 439-441)

terminalLine part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 439-441 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 440: return [`\${label}`, String(text || "").trim()).filter(Boolean).join("\n");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

439: function terminalLine(label, text = "") {
440:   return [`${label}`, String(text || "").trim()).filter(Boolean).join("\n");
441: }

```

processTerminalText (server.mjs, lines 443-451)

processTerminalText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 443-451 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, toFixed, map, trimEnd, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 450: return [status, output || (result.ok ? "Completed without diagnostic output." : "No compiler output was returned.")].join("\n");.

Important local values include elapsedSeconds at line 444; elapsed at line 445; status at line 446; output at line 449. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
443: function processTerminalText(result = {}) {
444:   const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
445:   const elapsed = elapsedSeconds ? ` in ${elapsedSeconds}s` : "";
446:   const status = result.timedOut
447:     ? `Timed out after ${elapsedSeconds} ? ${elapsedSeconds}s` : "the configured timeout"`
448:     : `Exit code ${result.code ?? "unknown"}${elapsed}`;
449:   const output = [result.stdout, result.stderr].map((part) => String(part ||
""))
.trimEnd()).filter(Boolean).join("\n");
450:   return [status, output || (result.ok ? "Completed without diagnostic output." : "No compiler output was
returned.")]
.join("\n");
451: }
```

pathVariantsForReplacement (server.mjs, lines 453-461)

pathVariantsForReplacement part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 453-461 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references from, Set, normalize, replace, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 455: return Array.from(new Set([.

Important local values include clean at line 454. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
453: function pathVariantsForReplacement(value = "") {
454:   const clean = String(value || "");
455:   return Array.from(new Set([
456:     clean,
457:     path.normalize(clean),
458:     clean.replace(/\\\/g, "/"),
459:     path.normalize(clean).replace(/\\\/g, "/")
460:   ]
.filter(Boolean)));
461: }
```

replacePathReferences (server.mjs, lines 463-474)

replacePathReferences part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 463-474 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pathVariantsForReplacement, split, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 473: return output;

Important local values include output at line 464; from at line 466; to at line 467. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
463: function replacePathReferences(text = "", replacements = []) {
464:   let output = String(text || "");
465:   for (const replacement of replacements) {
466:     const from = replacement?.from || "";
467:     const to = replacement?.to || "";
468:     if (!from || !to) continue;
469:     for (const variant of pathVariantsForReplacement(from)) {
470:       output = output.split(variant).join(to);
471:     }
472:   }
473:   return output;
474: }
```

processTerminalTextWithPaths (server.mjs, lines 476-478)

processTerminalTextWithPaths part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 476-478 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `replacePathReferences`, and `processTerminalText`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 477: `return replacePathReferences(processTerminalText(result), replacements);`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
476: function processTerminalTextWithPaths(result = {}, replacements = []) {
477:   return replacePathReferences(processTerminalText(result), replacements);
478: }
```

cFamilyCompileProfile (server.mjs, lines 480-501)

`cFamilyCompileProfile` part of the compile code language/tool/build/run flow. In this file, it appears around lines 480-501 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `extname`, `toLowerCase`, and `includes`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 484: `return {`.

Important local values include `isCpp` at line 481; `ext` at line 482; `header` at line 483. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
480: function cFamilyCompileProfile(language = "c", fileName = "main.c") {
481:   const isCpp = language === "cpp";
482:   const ext = path.extname(String(fileName || "")).toLowerCase();
483:   const header = [".h", ".hpp", ".hh", ".hxx"].includes(ext);
484:   return {
485:     compiler: isCpp ? "g++" : "gcc",
486:     standard: isCpp ? "-std=c++20" : "-std=c17",
487:     languageFlag: isCpp ? "c++" : "c",
488:     label: isCpp ? "C++" : "C",
489:     header,
490:     flags: [
491:       isCpp ? "-std=c++20" : "-std=c17",
492:       "-Wall",
493:       "-Wextra",
494:       "-Wpedantic",
495:       "-Wshadow",
496:       "-O0",
497:       "-g3",
498:       "-fdiagnostics-color=never"
499:     ]
500:   };
501: }
```

cFamilyBinaryName (server.mjs, lines 503-506)

`cFamilyBinaryName` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 503-506 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `parse`, and `safeSegment`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 505: `return `${safeSegment(parsed.name, "program")}.exe`;`.

Important local values include `parsed` at line 504. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
503: function cFamilyBinaryName(fileName = "program") {
504:   const parsed = path.parse(String(fileName || "program"));
505:   return `${safeSegment(parsed.name, "program")}.exe`;
```


506: }

toolVersionLine (server.mjs, lines 508-521)

toolVersionLine part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 508-521 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references has, get, runProcess, map, trim, filter, join, split, find, basename, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 509: if (!toolPath) return ""; line 510: if (compileToolVersionCache.has(toolPath)) return compileToolVersionCache.get(toolPath); line 520: return version;

Important local values include result at line 511; version at line 512. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
508: async function toolVersionLine(toolPath = "") {
509:   if (!toolPath) return "";
510:   if (compileToolVersionCache.has(toolPath)) return compileToolVersionCache.get(toolPath);
511:   const result = await runProcess(toolPath, ["--version"], { timeoutMs: 7000 });
512:   const version = [result.stdout, result.stderr]
513:     .map((part) => String(part || "").trim())
514:     .filter(Boolean)
515:     .join("\n")
516:     .split(/\r?\n/)
517:     .map((line) => line.trim())
518:     .find(Boolean) || path.basename(toolPath);
519:   compileToolVersionCache.set(toolPath, version);
520:   return version;
521: }
```

cFamilyRunOutput (server.mjs, lines 523-530)

cFamilyRunOutput part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 523-530 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, toFixed, map, trimEnd, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 529: return [status, output || (result.ok ? "Program completed without output." : "No program output was returned.")]

Important local values include elapsedSeconds at line 524; status at line 525; output at line 528. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
523: function cFamilyRunOutput(result = {}) {
524:   const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
525:   const status = result.timedOut
526:     ? `Program timed out after ${elapsedSeconds} s : "the configured timeout"`
527:     : `Program exit code ${result.code ?? "unknown"}${elapsedSeconds ? ` in ${elapsedSeconds} s` : ""}`;
528:   const output = [result.stdout, result.stderr].map((part) => String(part || ""))
529:     .trimEnd().filter(Boolean).join("\n");
529:   return [status, output || (result.ok ? "Program completed without output." : "No program output was returned.")]
530: }
```

cleanHdlSimulationOutput (server.mjs, lines 532-557)

cleanHdlSimulationOutput part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 532-557 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, trimEnd, filter, join, split, push, replace, isFinite, and toFixed. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 556: return `\${status}\n\${body}\${suffix}`;

Important local values include raw at line 533; lines at line 534; finishLines at line 535; signalLines at line 536; elapsedSeconds at line 544; status at line 545; body at line 548; suffix at line 555. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
532: function cleanHdlSimulationOutput(result = {}) {
533:   const raw = [result.stdout, result.stderr].map((part) => String(part ||
"").trimEnd()).filter(Boolean).join("\n");
534:   const lines = raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean);
535:   const finishLines = [];
536:   const signalLines = [];
537:   for (const line of lines) {
538:     if (/^$finish called at/i.test(line)) {
539:       finishLines.push(line.replace(/^.*?:\d+:\s*/, ""));
540:     } else {
541:       signalLines.push(line);
542:     }
543:   }
544:   const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
545:   const status = result.timedOut
546:     ? `Simulation timed out after ${elapsedSeconds} s` : "the configured timeout"
547:     : `Simulation exit code ${result.code ?? "unknown"}${elapsedSeconds} s` :
"";
548:   const body = signalLines.length
549:     ? signalLines.join("\n")
550:     : finishLines.length
551:     ? finishLines.join("\n")
552:     : result.ok
553:     ? "Simulation completed without printed output."
554:     : "No simulator output was returned.";
555:   const suffix = signalLines.length && finishLines.length ? `${finishLines.join("\n")}` : "";
556:   return `${status}\n${body}${suffix}`;
557: }
```

runProcess (server.mjs, lines 559-599)

This is the backend's controlled process runner. It launches a command with arguments, captures stdout and stderr, enforces a timeout, records elapsed time, and returns a structured result to the caller.

Git commands, compilers, simulators, and installers all run outside Node. This wrapper makes those risky external calls predictable and lets the UI show terminal output instead of freezing or silently failing.

It calls or references now, spawn, execFile, kill, on, toString, call, write, end, resolve, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 562: return new Promise((resolve) => {.

Important local values include timeoutMs at line 560; cwd at line 561; startedAt at line 563; child at line 564; stdout at line 570; stderr at line 571; timedOut at line 572; timer at line 573. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
559: async function runProcess(command, args = [], options = {}) {
560:   const timeoutMs = options.timeoutMs || 20000;
561:   const cwd = options.cwd || compileRoot;
562:   return new Promise((resolve) => {
563:     const startedAt = Date.now();
564:     const child = spawn(command, args, {
565:       cwd,
566:       env: { ...process.env, ...(options.env || {}) },
567:       shell: false,
568:       windowsHide: true
569:     });
570:     let stdout = "";
571:     let stderr = "";
572:     let timedOut = false;
573:     const timer = setTimeout(() => {
574:       timedOut = true;
575:       if (process.platform === "win32" && child.pid) {
576:         execFile("taskkill", ["/pid", String(child.pid), "/T", "/F"], { windowsHide: true }, () => {});
577:       }
578:       child.kill("SIGKILL");
579:     }, timeoutMs);
580:     child.stdout?.on("data", (chunk) => {
581:       stdout += chunk.toString();
582:     });
583:     child.stderr?.on("data", (chunk) => {
584:       stderr += chunk.toString();
585:     });
```

```

586:     if (Object.prototype.hasOwnProperty.call(options, "input")) {
587:       child.stdin?.write(String(options.input || ""));
588:     }
589:     child.stdin?.end();
590:     child.on("error", (error) => {
591:       clearTimeout(timer);
592:       resolve({ ok: false, code: -1, stdout, stderr: `${stderr}\n${error.message}`.trim(), timedOut,
elapsedMs: Date.now() - startedAt });
593:     });
594:     child.on("close", (code) => {
595:       clearTimeout(timer);
596:       resolve({ ok: !timedOut && code === 0, code, stdout, stderr, timedOut, elapsedMs: Date.now() -
startedAt });
597:     });
598:   });
599: }

```

compileToolStatus (server.mjs, lines 601-620)

compileToolStatus part of the compile code language/tool/build/run flow. In this file, it appears around lines 601-620 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, values, flatMap, findTool, entries, every, fromEntries, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 619: return { compileRoot, languages, tools };

Important local values include tools at line 602; uniqueTools at line 603; languages at line 607; required at line 609. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

601: async function compileToolStatus() {
602:   const tools = {};
603:   const uniqueTools = [...new Set(Object.values(compileLanguageProfiles).flatMap((profile) =>
profile.primaryTools))];
604:   for (const tool of uniqueTools) {
605:     tools[tool] = await findTool(tool);
606:   }
607:   const languages = {};
608:   for (const [id, profile] of Object.entries(compileLanguageProfiles)) {
609:     const required = profile.primaryTools;
610:     languages[id] = {
611:       label: profile.label,
612:       defaultFile: profile.defaultFile,
613:       extensions: profile.extensions,
614:       ready: required.every((tool) => tools[tool]),
615:       tools: Object.fromEntries(required.map((tool) => [tool, tools[tool] || "missing"])),
616:       installHints: profile.winget || []
617:     };
618:   }
619:   return { compileRoot, languages, tools };
620: }

```

isHdlLanguage (server.mjs, lines 622-624)

isHdlLanguage part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 622-624 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 623: return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

622: function isHdlLanguage(language = "") {
623:   return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));
624: }

```

inferCompileFileRole (server.mjs, lines 626-633)

inferCompileFileRole part of the compile code language/tool/build/run flow. In this file, it appears around lines 626-633 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isHdlLanguage, toLowerCase, b, and tb. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 627: if (!isHdlLanguage(language)) return "source";; line 630: return "testbench";; line 632: return "design";.

Important local values include haystack at line 628. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
626: function inferCompileFileRole(fileName = "", code = "", language = "") {
627:   if (!isHdlLanguage(language)) return "source";
628:   const haystack = `${fileName}\n${code}`.toLowerCase();
629:   if (/\\b(tb|testbench)\\b|(^|[-])tb([-]|\\.)|test[-]?bench|\\$dumpfile|\\$dumpvars|module\\s+tb[_a-z0-
9]*\\/i.test(haystack)) {
630:     return "testbench";
631:   }
632:   return "design";
633: }
```

normalizeCompileFileRole (server.mjs, lines 635-639)

normalizeCompileFileRole part of the compile code language/tool/build/run flow. In this file, it appears around lines 635-639 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isHdlLanguage, trim, toLowerCase, replace, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 636: if (!isHdlLanguage(language)) return "source";; line 638: return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";.

Important local values include clean at line 637. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
635: function normalizeCompileFileRole(value = "", language = "") {
636:   if (!isHdlLanguage(language)) return "source";
637:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
638:   return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";
639: }
```

saveCompileSource (server.mjs, lines 641-662)

saveCompileSource persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 641-662 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodeLanguage, detectCodeLanguageFromSource, safeSegment, safeCodeFileName, parse, resolveInsideCompileRoot, mkdir, writeFile, clampText, normalizeCompileFileRole, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 661: return metadata;.

Important local values include lang at line 642; projectFolder at line 643; codeFileName at line 644; fileFolder at line 645; folder at line 646; sourcePath at line 647; metadata at line 651. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
641: async function saveCompileSource({ projectId, fileId, title, fileName, language, role = "", code, stdin =
"" }) {
642:   const lang = normalizeCodeLanguage(language || detectCodeLanguageFromSource(code, fileName));
643:   const projectFolder = safeSegment(projectId, "project");
644:   const codeFileName = safeCodeFileName(fileName, lang);
645:   const fileFolder = safeSegment(fileId || path.parse(codeFileName).name, path.parse(codeFileName).name);
646:   const folder = resolveInsideCompileRoot(projectFolder, fileFolder);
647:   const sourcePath = resolveInsideCompileRoot(projectFolder, fileFolder, codeFileName);
```

```

648:   await mkdir(folder, { recursive: true });
649:   await writeFile(sourcePath, String(code || ""), "utf8");
650:   if (stdin) await writeFile(resolveInsideCompileRoot(projectFolder, fileFolder, "stdin.txt"),
String(stdin), "utf8");
651:   const metadata = {
652:     id: fileFolder,
653:     title: clampText(title || codeFileName, 180),
654:     fileName: codeFileName,
655:     language: lang,
656:     role: normalizeCompileFileRole(role || inferCompileFileRole(codeFileName, code, lang), lang),
657:     sourcePath,
658:     savedAt: new Date().toISOString()
659:   };
660:   await writeFile(resolveInsideCompileRoot(projectFolder, fileFolder, "compile-meta.json"),
`${JSON.stringify(metadata, null, 2)}\n`, "utf8");
661:   return metadata;
662: }

```

javaMainClassName (server.mjs, lines 664-670)

javaMainClassName part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 664-670 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references match, and parse. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 667: if (publicMatch) return publicMatch[1]; line 669: return classMatch?.[1] || path.parse(fileName).name || "Main";.

Important local values include source at line 665; publicMatch at line 666; classMatch at line 668. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

664: function javaMainClassName(code = "", fileName = "Main.java") {
665:   const source = String(code || "");
666:   const publicMatch = source.match(/bpublic\s+class\s+([A-Za-z_$][\w$]*)/);
667:   if (publicMatch) return publicMatch[1];
668:   const classMatch = source.match(/bclass\s+([A-Za-z_$][\w$]*)/);
669:   return classMatch?.[1] || path.parse(fileName).name || "Main";
670: }

```

compileCacheKey (server.mjs, lines 672-677)

compileCacheKey part of the compile code language/tool/build/run flow. In this file, it appears around lines 672-677 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createHash, update, stringify, digest, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 673: return createHash("sha256").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

672: function compileCacheKey(parts = {}) {
673:   return createHash("sha256")
674:     .update(JSON.stringify(parts))
675:     .digest("hex")
676:     .slice(0, 24);
677: }

```

compileCacheDirectory (server.mjs, lines 679-681)

compileCacheDirectory part of the compile code language/tool/build/run flow. In this file, it appears around lines 679-681 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `resolveInsideCompileRoot`, and `safeSegment`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 680: `return resolveInsideCompileRoot(safeSegment(projectId, "project"), ".build-cache", safeSegment(language, "language"), key);`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
679: function compileCacheDirectory(projectId, language, key) {
680:   return resolveInsideCompileRoot(safeSegment(projectId, "project"), ".build-cache",
safeSegment(language, "language"), key);
681: }
```

cachedBuildLine (server.mjs, lines 683-685)

`cachedBuildLine` reads, writes, or validates cached state. In this file, it appears around lines 683-685 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 684: `return `${type} cache hit: using existing artifact\n${outputPath}`;`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
683: function cachedBuildLine(type, outputPath) {
684:   return `${type} cache hit: using existing artifact\n${outputPath}`;
685: }
```

hdlModuleNames (server.mjs, lines 687-689)

`hdlModuleNames` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 687-689 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `matchAll`, and `map`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 688: `return [...String(code || "").matchAll(/bmodule\s+([A-Za-z_$][\w$]*)/g)].map((match) => match[1]);`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
687: function hdlModuleNames(code = "") {
688:   return [...String(code || "").matchAll(/bmodule\s+([A-Za-z_$][\w$]*)/g)].map((match) => match[1]);
689: }
```

hdlHasWaveDump (server.mjs, lines 691-694)

`hdlHasWaveDump` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 691-694 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 693: `return /^$dumpfile\s*(/.test(source) && /^$dumpvars\s*(/.test(source);`.

Important local values include source at line 692. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
691: function hdlHasWaveDump(code = "") {
692:   const source = String(code || "");
693:   return /\$dumpfile\s*(\/.test(source) && /\$dumpvars\s*(\/.test(source);
694: }
```

hdlFilesFromPayload (server.mjs, lines 696-727)

hdlFilesFromPayload part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 696-727 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, Map, normalizeCodeLanguage, detectCodeLanguageFromSource, isHdlLanguage, trim, safeCodeFileName, safeSegment, parse, set, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 702: if (!isHdlLanguage(language) || !code.trim()) return;; line 723: return [...byId.values()].sort((a, b) => {; line 724: if (a.role === b.role) return a.fileName.localeCompare(b.fileName);; line 725: return a.role === "testbench" ? 1 : -1;;

Important local values include incoming at line 697; byId at line 698; addFile at line 699; code at line 700; language at line 701; fileName at line 703; id at line 704. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
696: function hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog") {
697:   const incoming = Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : [];
698:   const byId = new Map();
699:   const addFile = (item = {}, fallbackId = "") => {
700:     const code = String(item.code || "");
701:     const language = normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code,
item.fileName));
702:     if (!isHdlLanguage(language) || !code.trim()) return;
703:     const fileName = safeCodeFileName(item.fileName || activeFileName ||
compileLanguageProfiles[language].defaultFile, language);
704:     const id = safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name);
705:     byId.set(id, {
706:       id,
707:       title: clampText(item.title || fileName, 180),
708:       fileName,
709:       language,
710:       role: normalizeCompileFileRole(item.role || inferCompileFileRole(fileName, code, language),
language),
711:       code
712:     });
713:   };
714:   incoming.forEach((item, index) => addFile(item, `hdl-${index + 1}`));
715:   addFile({
716:     id: payload.fileId,
717:     title: payload.title,
718:     fileName: activeFileName,
719:     language: activeLanguage,
720:     role: payload.role,
721:     code: payload.code
722:   }, "active");
723:   return [...byId.values()].sort((a, b) => {
724:     if (a.role === b.role) return a.fileName.localeCompare(b.fileName);
725:     return a.role === "testbench" ? 1 : -1;
726:   });
727: }
```

compileActionFromPayload (server.mjs, lines 729-733)

compileActionFromPayload part of the compile code language/tool/build/run flow. In this file, it appears around lines 729-733 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, replace, includes, and isHdlLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 731: if (!["compile", "build", "run", "simulate"].includes(clean)) return clean;; line 732: return isHdlLanguage(language) ? "simulate" : "run";.

Important local values include clean at line 730. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
729: function compileActionFromPayload(value = "", language = "") {
730:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "-");
731:   if (["compile", "build", "run", "simulate"].includes(clean)) return clean;
732:   return isHdlLanguage(language) ? "simulate" : "run";
733: }
```

compileActionLabel (server.mjs, lines 735-740)

compileActionLabel part of the compile code language/tool/build/run flow. In this file, it appears around lines 735-740 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isHdlLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 736: if (action === "compile") return "Compile"; line 737: if (action === "build") return "Build project"; line 738: if (action === "simulate") return "Simulate"; line 739: return isHdlLanguage(language) ? "Simulate" : "Run";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
735: function compileActionLabel(action = "run", language = "") {
736:   if (action === "compile") return "Compile";
737:   if (action === "build") return "Build project";
738:   if (action === "simulate") return "Simulate";
739:   return isHdlLanguage(language) ? "Simulate" : "Run";
740: }
```

compileWorkspaceFilesFromPayload (server.mjs, lines 742-770)

compileWorkspaceFilesFromPayload part of the compile code language/tool/build/run flow. In this file, it appears around lines 742-770 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, Map, trim, normalizeCodeLanguage, detectCodeLanguageFromSource, safeCodeFileName, safeSegment, parse, set, clampText, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 747: if (!code.trim()) return;; line 769: return [...byId.values()].sort((a, b) => a.fileName.localeCompare(b.fileName));.

Important local values include incoming at line 743; byId at line 744; addFile at line 745; code at line 746; language at line 748; fileName at line 749; id at line 750. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
742: function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage =
"javascript") {
743:   const incoming = Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : [];
744:   const byId = new Map();
745:   const addFile = (item = {}, fallbackId = "") => {
746:     const code = String(item.code || "");
747:     if (!code.trim()) return;
748:     const language = normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code,
item.fileName));
749:     const fileName = safeCodeFileName(item.fileName || activeFileName ||
compileLanguageProfiles[language]?.defaultFile || "source.txt", language);
750:     const id = safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name);
751:     byId.set(id, {
752:       id,
753:       title: clampText(item.title || fileName, 180),
754:       fileName,
755:       language,
756:       role: normalizeCompileFileRole(item.role || inferCompileFileRole(fileName, code, language),
language),
757:       code
758:     });
759:   };
760:   incoming.forEach((item, index) => addFile(item, `workspace-${index + 1}`));
761:   addFile({
```



```

762:     id: payload.fileId,
763:     title: payload.title,
764:     fileName: activeFileName,
765:     language: activeLanguage,
766:     role: payload.role,
767:     code: payload.code
768:   }, "active");
769:   return [...byId.values()].sort((a, b) => a.fileName.localeCompare(b.fileName));
770: }

```

writeCompileWorkspaceSources (server.mjs, lines 772-788)

writeCompileWorkspaceSources persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 772-788 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mkdir, Map, includes, normalizeCodeLanguage, extname, toLowerCase, parse, get, set, join, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 787: return written;.

Important local values include seen at line 774; written at line 775; parsed at line 779; count at line 780; uniqueName at line 782; sourcePath at line 783. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

772: async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {}) {
773:   await mkdir(targetDir, { recursive: true });
774:   const seen = new Map();
775:   const written = [];
776:   for (const file of files) {
777:     if (options.languages?.length && !options.languages.includes(normalizeCodeLanguage(file.language)))
continue;
778:     if (options.extensions?.length &&
!options.extensions.includes(path.extname(file.fileName).toLowerCase())) continue;
779:     const parsed = path.parse(file.fileName);
780:     const count = seen.get(file.fileName) || 0;
781:     seen.set(file.fileName, count + 1);
782:     const uniqueName = count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName;
783:     const sourcePath = path.join(targetDir, uniqueName);
784:     await writeFile(sourcePath, file.code, "utf8");
785:     written.push({ ...file, uniqueName, sourcePath });
786:   }
787:   return written;
788: }

```

sourceDisplayName (server.mjs, lines 790-792)

sourceDisplayName part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 790-792 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references basename. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 791: return file.uniqueName || file.fileName || path.basename(file.sourcePath || "source");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

790: function sourceDisplayName(file = {}) {
791:   return file.uniqueName || file.fileName || path.basename(file.sourcePath || "source");
792: }

```

cFamilyWorkspaceSources (server.mjs, lines 794-803)

cFamilyWorkspaceSources part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 794-803 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, filter, has, extname, toLowerCase, some, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 802: return selected;.

Important local values include exts at line 795; selected at line 798. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
794: function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "") {
795:   const exts = language === "cpp"
796:     ? new Set([".cpp", ".cc", ".cxx", ".c++", ".hpp", ".hh", ".hxx", ".h"])
797:     : new Set([".c", ".h"]);
798:   const selected = files.filter((file) => exts.has(path.extname(file.fileName).toLowerCase()));
799:   if (!selected.some((file) => file.fileName === activeFileName)) {
800:     selected.push(...files.filter((file) => file.fileName === activeFileName));
801:   }
802:   return selected;
803: }
```

writeHdlSimulationSources (server.mjs, lines 805-821)

writeHdlSimulationSources persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 805-821 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join, rm, mkdir, Map, parse, get, set, writeFile, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 820: return written;.

Important local values include sourceDir at line 806; seen at line 809; written at line 810; parsed at line 812; count at line 813; uniqueName at line 815; sourcePath at line 816. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
805: async function writeHdlSimulationSources(files = [], cacheDir = "") {
806:   const sourceDir = path.join(cacheDir, "sources");
807:   await rm(sourceDir, { recursive: true, force: true });
808:   await mkdir(sourceDir, { recursive: true });
809:   const seen = new Map();
810:   const written = [];
811:   for (const file of files) {
812:     const parsed = path.parse(file.fileName);
813:     const count = seen.get(file.fileName) || 0;
814:     seen.set(file.fileName, count + 1);
815:     const uniqueName = count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName;
816:     const sourcePath = path.join(sourceDir, uniqueName);
817:     await writeFile(sourcePath, file.code, "utf8");
818:     written.push({ ...file, uniqueName, sourcePath });
819:   }
820:   return written;
821: }
```

findFilesByExtension (server.mjs, lines 823-836)

findFilesByExtension part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 823-836 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pathExists, readdir, join, isDirectory, push, isFile, toLowerCase, and endsWith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 824: if (!folder || maxDepth < 0 || !(await pathExists(folder))) return []; line 835: return matches;.

Important local values include entries at line 825; matches at line 826; fullPath at line 828. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

823: async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3) {
824:   if (!folder || maxDepth < 0 || !(await pathExists(folder))) return [];
825:   const entries = await readdir(folder, { withFileTypes: true }).catch(() => []);
826:   const matches = [];
827:   for (const entry of entries) {
828:     const fullPath = path.join(folder, entry.name);
829:     if (entry.isDirectory()) {
830:       matches.push(...await findFilesByExtension(fullPath, extension, maxDepth - 1));
831:     } else if (entry.isFile() && entry.name.toLowerCase().endsWith(extension.toLowerCase())) {
832:       matches.push(fullPath);
833:     }
834:   }
835:   return matches;
836: }

```

normalizeVcdValue (server.mjs, lines 838-844)

normalizeVcdValue validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 838-844 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, slice, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 840: if (!clean) return "x"; line 841: if (/^[01xz]\$/i.test(clean)) return clean.toLowerCase(); line 842: if (/^[br][01xz_]+\$/i.test(clean)) return clean.slice(1).replace(/_/g, "").toLowerCase(); line 843: return clean.toLowerCase();.

Important local values include clean at line 839. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

838: function normalizeVcdValue(raw = "") {
839:   const clean = String(raw || "").trim();
840:   if (!clean) return "x";
841:   if (/^[01xz]$/i.test(clean)) return clean.toLowerCase();
842:   if (/^[br][01xz_]+$/i.test(clean)) return clean.slice(1).replace(/_/g, "").toLowerCase();
843:   return clean.toLowerCase();
844: }

```

parseVcdScopeText (server.mjs, lines 846-933)

This function parses VCD waveform text emitted by HDL simulations. It reads signal definitions, time markers, value changes, scopes, and symbols, then converts the raw waveform file into structured signal data.

The scope UI cannot draw from raw VCD text directly. This parser is what lets a SystemVerilog or Verilog simulation become a visible timing waveform in the builder.

It calls or references split, Map, trim, startsWith, replace, push, includes, match, pop, has, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 927: return {.

Important local values include lines at line 847; scopes at line 848; signalsByCode at line 849; timeScaleParts at line 850; inTimescale at line 851; time at line 852; maxTime at line 853; definitionDone at line 854; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

846: function parseVcdScopeText(text = "", source = "waveform.vcd") {
847:   const lines = String(text || "").split(/\r?\n/);
848:   const scopes = [];
849:   const signalsByCode = new Map();
850:   const timeScaleParts = [];
851:   let inTimescale = false;
852:   let time = 0;
853:   let maxTime = 0;
854:   let definitionDone = false;
855:   for (const rawLine of lines) {
856:     const line = rawLine.trim();
857:     if (!line) continue;
858:     if (line.startsWith("$timescale")) {
859:       inTimescale = true;
860:       const inline = line.replace("$timescale", "").replace("$end", "").trim();
861:       if (inline) timeScaleParts.push(inline);
862:       continue;
863:     }
864:     if (inTimescale) {
865:       if (line === "$end") {

```

```

866:         inTimescale = false;
867:     } else {
868:         timeScaleParts.push(line.replace("$end", "").trim());
869:         if (line.includes("$end")) inTimescale = false;
870:     }
871:     continue;
872: }
873: const scopeMatch = line.match(/^\$scope\$s+\$s+(.?)\$s+\$end$/);
874: if (scopeMatch) {
875:     scopes.push(scopeMatch[1]);
876:     continue;
877: }
878: if (/^\$supscope\$b/.test(line)) {
879:     scopes.pop();
880:     continue;
881: }
882: const varMatch = line.match(/^\$var\$s+\$s+(\d+)\$s+(\$s+)\$s+(.?)\$s+\$end$/);
883: if (varMatch) {
884:     const [, width, code, reference] = varMatch;
885:     if (!signalsByCode.has(code) && signalsByCode.size < 64) {
886:         const cleanReference = reference.replace(/\$s+\[^\$]\$/, "");
887:         signalsByCode.set(code, {
888:             code,
889:             name: [...scopes, cleanReference].filter(Boolean).join("."),
890:             reference: cleanReference,
891:             width: Number(width) || 1,
892:             changes: []
893:         });
894:     }
895:     continue;
896: }
897: if (line.startsWith("$enddefinitions")) {
898:     definitionDone = true;
899:     continue;
900: }
901: if (!definitionDone) continue;
902: if (line.startsWith("#")) {
903:     time = Number(line.slice(1)) || 0;
904:     maxTime = Math.max(maxTime, time);
905:     continue;
906: }
907: let code = "";
908: let value = "";
909: const vectorMatch = line.match(/^\([br][01xz_]+\)\$s+(\$s+)\$/i);
910: if (vectorMatch) {
911:     value = normalizeVcdValue(vectorMatch[1]);
912:     code = vectorMatch[2];
913: } else if (/^\[01xz]/i.test(line)) {
914:     value = normalizeVcdValue(line[0]);
915:     code = line.slice(1).trim();
916: }
917: const signal = signalsByCode.get(code);
918: if (!signal || !value) continue;
919: const previous = signal.changes[signal.changes.length - 1];
920: if (!previous || previous.value !== value || previous.time !== time) {
921:     if (signal.changes.length < 600) signal.changes.push({ time, value });
922: }
923: }
924: const signals = [...signalsByCode.values()]
925:     .filter((signal) => signal.changes.length)
926:     .slice(0, 32);
927: return {
928:     source: path.basename(source),
929:     timeScale: timeScaleParts.join(" ").replace(/\$s+/g, " ").trim() || "ticks",
930:     maxTime,
931:     signals
932: };
933: }

```

readHdlWaveform (server.mjs, lines 935-942)

This function searches the compile cache for generated VCD waveform files, selects an appropriate waveform, and runs it through the VCD parser.

A simulator may generate files in a build directory rather than returning all useful information through stdout. This function lets the builder recover waveform evidence after simulation finishes.

It calls or references `findFilesByExtension`, `sort`, `readFile`, `slice`, and `parseVcdScopeText`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 937: `if (!vcdFiles.length) return null;`; line 941: `return parseVcdScopeText(limited, selected);`.

Important local values include `vcdFiles` at line 936; `selected` at line 938; `text` at line 939; `limited` at line 940. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

935: async function readHdlWaveform(cacheDir = "") {
936:   const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);
937:   if (!vcdFiles.length) return null;
938:   const selected = vcdFiles.sort((a, b) => a.length - b.length)[0];
939:   const text = await readFile(selected, "utf8");
940:   const limited = text.length > 6_000_000 ? text.slice(0, 6_000_000) : text;
941:   return parseVcdScopeText(limited, selected);
942: }

```

clearHdlWaveforms (server.mjs, lines 944-947)

clearHdlWaveforms part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 944-947 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references findFilesByExtension, all, map, and rm. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include vcdFiles at line 945. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

944: async function clearHdlWaveforms(cacheDir = "") {
945:   const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);
946:   await Promise.all(vcdFiles.map((filePath) => rm(filePath, { force: true })).catch(() => {}));
947: }

```

compileAndRunCode (server.mjs, lines 949-1319)

This function is the core Compile Code engine. It saves or gathers workspace files, detects the requested language/action, checks required tools, compiles when needed, runs when requested, handles HDL simulation/testbench requirements, reads generated waveform files, and returns terminal output plus metadata.

It turns the builder's code workspace into a small IDE. Keeping this logic in one backend function lets the frontend stay focused on editing while the backend owns language-specific build behavior.

It calls or references trim, toLowerCase, normalizeCodeLanguage, detectCodeLanguageFromSource, safeCodeFileName, saveCompileSource, safeSegment, resolveInsideCompileRoot, compileActionFromPayload, compileWorkspaceFilesFromPayload, and 20 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 966: if (runDir && runSourcePath) return { runDir, runSourcePath, writtenSources: runWorkspaceSources };; line 990: return { runDir, runSourcePath, writtenSources: runWorkspaceSources };; line 1003: return { found, missingTools };; line 1010: return {}. There are 8 additional return statements in the function body.

Important local values include requestedLanguage at line 950; language at line 951; profile at line 954; fileName at line 955; saved at line 956; projectFolder at line 957; sourcePath at line 958; sourceCode at line 959; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

949: async function compileAndRunCode(payload = {}) {
950:   const requestedLanguage = String(payload.language || "").trim().toLowerCase();
951:   const language = requestedLanguage && requestedLanguage !== "auto"
952:     ? normalizeCodeLanguage(requestedLanguage)
953:     : detectCodeLanguageFromSource(payload.code, payload.fileName);
954:   const profile = compileLanguageProfiles[language] || compileLanguageProfiles.javascript;
955:   const fileName = safeCodeFileName(payload.fileName, language);
956:   const saved = await saveCompileSource({ ...payload, language, fileName });
957:   const projectFolder = safeSegment(payload.projectId, "project");
958:   const sourcePath = resolveInsideCompileRoot(projectFolder, safeSegment(saved.id), saved.fileName);
959:   const sourceCode = String(payload.code || "");
960:   const action = compileActionFromPayload(payload.action, language);
961:   const allWorkspaceFiles = compileWorkspaceFilesFromPayload(payload, saved.fileName, language);
962:   let runDir = "";
963:   let runSourcePath = "";
964:   let runWorkspaceSources = [];
965:   const ensureRunSource = async (options = {}) => {
966:     if (runDir && runSourcePath) return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
967:     const runId = `${Date.now()}-${Math.random().toString(16).slice(2)}`;
968:     runDir = resolveInsideCompileRoot(projectFolder, ".runs", runId);
969:     await mkdir(runDir, { recursive: true });
970:     const filesToWrite = Array.isArray(options.files) && options.files.length ? options.files :
allWorkspaceFiles;

```

```

971:     runWorkspaceSources = await writeCompileWorkspaceSources(filesToWrite, runDir, {
972:       languages: options.languages,
973:       extensions: options.extensions
974:     });
975:     const active = runWorkspaceSources.find((file) => file.id === saved.id || file.fileName ===
saved.fileName);
976:     runSourcePath = active?.sourcePath || path.join(runDir, saved.fileName);
977:     if (!active) {
978:       await cp(sourcePath, runSourcePath, { force: true });
979:       runWorkspaceSources.push({
980:         id: saved.id,
981:         title: saved.title,
982:         fileName: saved.fileName,
983:         language,
984:         role: saved.role,
985:         code: sourceCode,
986:         uniqueName: saved.fileName,
987:         sourcePath: runSourcePath
988:       });
989:     }
990:     return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
991:   };
992:   const stdin = String(payload.stdin || "");
993:   const forceRebuild = payload.forceRebuild === true;
994:   const terminal = [];
995:   terminal.push(`${compileActionLabel(action, language)} request for ${saved.fileName}`);
996:   const append = (label, result) => {
997:     terminal.push(terminalLine(label, processTerminalText(result)));
998:   };
999:   const missing = async (tools) => {
1000:     const found = {};
1001:     for (const tool of tools) found[tool] = await findTool(tool);
1002:     const missingTools = tools.filter((tool) => !found[tool]);
1003:     return { found, missingTools };
1004:   };
1005:
1006:   if (language === "html") {
1007:     const openTags = (String(payload.code || "").match(/<[a-z][\w:-]*\b[^>*/>/gi) || []).length;
1008:     const closeTags = (String(payload.code || "").match(/<\/[a-z][\w:-]*>/gi) || []).length;
1009:     const ok = openTags >= closeTags;
1010:     return {
1011:       ok,
1012:       language,
1013:       saved,
1014:       terminal: [
1015:         "HTML validation pass",
1016:         ok ? "No obvious tag-balance issue was detected." : "The file may have more closing tags than
opening tags."
1017:       ],
1018:       `Saved source: ${saved.sourcePath}`
1019:     }.join("\n");
1020:   }
1021:
1022:   const stdinOptions = stdin ? { input: stdin } : {};
1023:   let ok = false;
1024:
1025:   if (language === "javascript") {
1026:     const node = await findTool("node");
1027:     if (!node) return { ok: false, language, saved, terminal: "Node.js was not found. Install Node.js to
run JavaScript." };
1028:     const jsFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) ===
"javascript");
1029:     const runSource = await ensureRunSource({ files: jsFiles.length ? jsFiles : allWorkspaceFiles,
languages: ["javascript"] });
1030:     const targets = action === "build"
1031:       ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "javascript")
1032:       : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
1033:     ok = true;
1034:     for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName:
saved.fileName }]) {
1035:       const check = await runProcess(node, ["--check", target.sourcePath], { cwd: runSource.runDir,
timeoutMs: 15000 });
1036:       terminal.push(terminalLine(`${path.basename(node)} --check ${sourceDisplayName(target)}`,
processTerminalText(check)));
1037:       ok = ok && check.ok;
1038:     }
1039:     if (action === "compile" || action === "build") {
1040:       terminal.push(ok ? "JavaScript syntax check completed." : "JavaScript syntax check found errors.");
1041:     } else if (ok) {
1042:       const run = await runProcess(node, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs:
20000, ...stdinOptions });
1043:       append(`${path.basename(node)} ${saved.fileName}`, run);
1044:       ok = run.ok;
1045:     }
1046:   } else if (language === "python") {
1047:     const python = await findTool("python");
1048:     if (!python) return { ok: false, language, saved, terminal: "Python was not found. Install Python to
run Python code." };
1049:     const pyFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) ===
"python");
1050:     const runSource = await ensureRunSource({ files: pyFiles.length ? pyFiles : allWorkspaceFiles,
languages: ["python"] });

```

```

1051:     const targets = action === "build"
1052:       ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "python")
1053:       : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
1054:     ok = true;
1055:     for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName:
saved.fileName }]) {
1056:       const check = await runProcess(python, ["-m", "py_compile", target.sourcePath], { cwd:
runSource.runDir, timeoutMs: 15000 });
1057:       terminal.push(terminalLine(`${path.basename(python)} -m py_compile ${sourceDisplayName(target)}`,
processTerminalText(check)));
1058:       ok = ok && check.ok;
1059:     }
1060:     if (action === "compile" || action === "build") {
1061:       terminal.push(ok ? "Python bytecode compile check completed." : "Python compile check found
errors.");
1062:     } else if (ok) {
1063:       const run = await runProcess(python, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs:
20000, ...stdinOptions });
1064:       append(`${path.basename(python)} ${saved.fileName}`, run);
1065:       ok = run.ok;
1066:     }
1067:   } else if (language === "c" || language === "cpp") {
1068:     const cProfile = cFamilyCompileProfile(language, saved.fileName);
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

append (server.mjs, lines 996-998)

append part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 996-998 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, terminalLine, and processTerminalText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include append at line 996. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

996:     const append = (label, result) => {
997:       terminal.push(terminalLine(label, processTerminalText(result)));
998:     };

```

missing (server.mjs, lines 999-1004)

missing part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 999-1004 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references async, findTool, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1003: return { found, missingTools };

Important local values include missing at line 999; found at line 1000; missingTools at line 1002. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

999:     const missing = async (tools) => {
1000:       const found = {};
1001:       for (const tool of tools) found[tool] = await findTool(tool);
1002:       const missingTools = tools.filter((tool) => !found[tool]);
1003:       return { found, missingTools };
1004:     };

```

installCompilerTools (server.mjs, lines 1321-1352)

installCompilerTools part of the compile code language/tool/build/run flow. In this file, it appears around lines 1321-1352 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizeCodeLanguage`, `findTool`, `runProcess`, `push`, `terminalLine`, `join`, `processTerminalText`, `clear`, and `compileToolStatus`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1325: `return {}`; line 1351: `return { ok, language: lang, terminal: terminal.join("\n\n"), tools: await compileToolStatus() };`.

Important local values include `lang` at line 1322; `profile` at line 1323; `winget` at line 1331; `terminal` at line 1332; `ok` at line 1333; `args` at line 1335; `result` at line 1346. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1321: async function installCompilerTools(language = "") {
1322:   const lang = normalizeCodeLanguage(language);
1323:   const profile = compileLanguageProfiles[lang] || compileLanguageProfiles.javascript;
1324:   if (!profile.winget?.length) {
1325:     return {
1326:       ok: false,
1327:       language: lang,
1328:       terminal: `${profile.label} does not have an automatic Winget compiler install configured. Install
the required tools manually, then restart the builder.`
1329:     };
1330:   }
1331:   const winget = await findTool("winget") || "winget";
1332:   const terminal = [];
1333:   let ok = true;
1334:   for (const packageId of profile.winget) {
1335:     const args = [
1336:       "install",
1337:       "--source",
1338:       "winget",
1339:       "--accept-source-agreements",
1340:       "--accept-package-agreements",
1341:       "--silent",
1342:       "--id",
1343:       packageId,
1344:       "--exact"
1345:     ];
1346:     const result = await runProcess(winget, args, { cwd: root, timeoutMs: 600000 });
1347:     terminal.push(terminalLine(`winget ${args.join(" ")}`, processTerminalText(result)));
1348:     ok = ok && result.ok;
1349:   }
1350:   compileToolCache.clear();
1351:   return { ok, language: lang, terminal: terminal.join("\n\n"), tools: await compileToolStatus() };
1352: }
```

sourceLooksTextual (server.mjs, lines 1354-1360)

`sourceLooksTextual` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1354-1360 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `toLowerCase`, and `includes`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1357: `if ([ "text", "webpage", "github", "linkedin", "public_profile", "resume_text", "code" ].includes(kind)) return true;`; line 1358: `return /\.(txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|tsx|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb|ino|xdc|sdc|tcl)$/i.test(url)`.

Important local values include `url` at line 1355; `kind` at line 1356. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1354: function sourceLooksTextual(source = {}) {
1355:   const url = String(source.url || "");
1356:   const kind = String(source.kind || "").toLowerCase();
1357:   if ([ "text", "webpage", "github", "linkedin", "public_profile", "resume_text", "code" ].includes(kind))
return true;
1358:   return /\.(txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|tsx|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb|ino|xdc|sdc|tcl)$/i.test(url)
1359:   || /github\.com|raw\.githubusercontent\.com/i.test(url);
1360: }
```

sourceUrlAllowed (server.mjs, lines 1362-1379)

`sourceUrlAllowed` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1362-1379 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level

documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URL, toLowerCase, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1366: return [; line 1377: return false;.

Important local values include parsed at line 1364; hostName at line 1365. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1362: function sourceUrlAllowed(url) {
1363:   try {
1364:     const parsed = new URL(url);
1365:     const hostName = parsed.hostname.toLowerCase();
1366:     return [
1367:       "localhost",
1368:       "127.0.0.1",
1369:       "github.com",
1370:       "api.github.com",
1371:       "raw.githubusercontent.com",
1372:       "gist.githubusercontent.com",
1373:       "linkedin.com",
1374:       "www.linkedin.com"
1375:     ].includes(hostName);
1376:   } catch {
1377:     return false;
1378:   }
1379: }
```

gitHubHeaders (server.mjs, lines 1384-1389)

gitHubHeaders part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1384-1389 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1385: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1384: function gitHubHeaders(accept = "application/vnd.github+json") {
1385:   return {
1386:     "Accept": accept,
1387:     "User-Agent": "OMB-Portfolio-Builder-AI"
1388:   };
1389: }
```

parseGitHubSourceUrl (server.mjs, lines 1391-1418)

parseGitHubSourceUrl part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1391-1418 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URL, toLowerCase, split, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 1397: return {}; line 1405: if (host !== "github.com" || !parts.length) return null;; line 1406: if (parts.length === 1) return { type: "profile", owner: parts[0] };; line 1409: return { ...base, type: "file", branch: parts[3], filePath: parts.slice(4).join("/") };. There are 3 additional return statements in the function body.

Important local values include parsed at line 1393; host at line 1394; parts at line 1395; base at line 1407. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1391: function parseGitHubSourceUrl(url) {
1392:   try {
```

```

1393:     const parsed = new URL(url);
1394:     const host = parsed.hostname.toLowerCase();
1395:     const parts = parsed.pathname.split("/").filter(Boolean);
1396:     if (host === "raw.githubusercontent.com" && parts.length >= 4) {
1397:       return {
1398:         type: "file",
1399:         owner: parts[0],
1400:         repo: parts[1],
1401:         branch: parts[2],
1402:         filePath: parts.slice(3).join("/")
1403:       };
1404:     }
1405:     if (host !== "github.com" || !parts.length) return null;
1406:     if (parts.length === 1) return { type: "profile", owner: parts[0] };
1407:     const base = { owner: parts[0], repo: parts[1] };
1408:     if (parts[2] === "blob" && parts.length >= 5) {
1409:       return { ...base, type: "file", branch: parts[3], filePath: parts.slice(4).join("/") };
1410:     }
1411:     if (parts[2] === "tree" && parts.length >= 4) {
1412:       return { ...base, type: "repo", branch: parts[3] };
1413:     }
1414:     return { ...base, type: "repo" };
1415:   } catch {
1416:     return null;
1417:   }
1418: }

```

fetchGitHubJson (server.mjs, lines 1420-1424)

fetchGitHubJson loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1420-1424 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, gitHubHeaders, and json. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1422: if (!response.ok) return null;; line 1423: return response.json().catch(() => null);.

Important local values include response at line 1421. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1420: async function fetchGitHubJson(url) {
1421:   const response = await fetch(url, { headers: gitHubHeaders() });
1422:   if (!response.ok) return null;
1423:   return response.json().catch(() => null);
1424: }

```

fetchLimitedText (server.mjs, lines 1426-1434)

fetchLimitedText loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1426-1434 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, gitHubHeaders, clampText, text, get, and stripHtmlToText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1430: if (!response.ok) return "";; line 1433: return /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;.

Important local values include response at line 1427; rawText at line 1431; contentType at line 1432. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1426: async function fetchLimitedText(url, maxLength = 12000) {
1427:   const response = await fetch(url, {
1428:     headers: gitHubHeaders("text/plain,text/markdown,text/html,application/json;q=0.8,*/*;q=0.1")
1429:   });
1430:   if (!response.ok) return "";
1431:   const rawText = clampText(await response.text(), maxLength);
1432:   const contentType = response.headers.get("Content-Type") || "";
1433:   return /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;
1434: }

```

githubQuestionTokens (server.mjs, lines 1436-1442)

githubQuestionTokens part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1436-1442 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, toLowerCase, replace, split, filter, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1437: return [...new Set(String(question || ""))].

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1436: function githubQuestionTokens(question = "") {
1437:   return [...new Set(String(question || ""))
1438:     .toLowerCase()
1439:     .replace(/^[a-z0-9_#\+\.\s-]+/g, " ")
1440:     .split(/\s+/)
1441:     .filter((token) => token.length >= 3 && ![ "the", "and", "with", "from", "show", "code", "file",
"github"].includes(token))];
1442: }
```

scoreGitHubFile (server.mjs, lines 1444-1455)

scoreGitHubFile part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1444-1455 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, githubQuestionTokens, b, forEach, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1454: return score;.

Important local values include cleanPath at line 1445; tokens at line 1446; score at line 1447. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1444: function scoreGitHubFile(filePath = "", question = "") {
1445:   const cleanPath = filePath.toLowerCase();
1446:   const tokens = githubQuestionTokens(question);
1447:   let score = githubTextFilePattern.test(cleanPath) ? 10 : 0;
1448:   if (/readme\.md$/i.test(cleanPath)) score += 55;
1449:   if (/\/(c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|ino|tcl|xdc|sdc)$/i.test(cleanPath)) score += 28;
1450:   if (/\/(test|tb|bench|sim|simulation|src|firmware|hardware|rtl|driver|main)\b/i.test(cleanPath)) score
+= 12;
1451:   tokens.forEach((token) => {
1452:     if (cleanPath.includes(token)) score += 18;
1453:   });
1454:   return score;
1455: }
```

wantsGitHubCode (server.mjs, lines 1457-1459)

wantsGitHubCode part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1457-1459 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1458: return /\b(code|source|snippet|implementation|firmware|driver|module|verilog|vhd|python|javascript|typescript|c\+\+|cpp|c\s+code|pull|show|display)\b/i.test(question);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1457: function wantsGitHubCode(question = "") {
1458:   return /\b(code|source|snippet|implementation|firmware|driver|module|verilog|vhd|python|javascript|typ
escript|c\+\+|cpp|c\s+code|pull|show|display)\b/i.test(question);
1459: }
```

scoreGitHubRepo (server.mjs, lines 1461-1481)

scoreGitHubRepo part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1461-1481 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references githubQuestionTokens, isArray, join, toLowerCase, forEach, includes, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1480: return score;.

Important local values include tokens at line 1462; haystack at line 1463; score at line 1470. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1461: function scoreGitHubRepo(repo = {}, question = "") {
1462:   const tokens = githubQuestionTokens(question);
1463:   const haystack = [
1464:     repo.full_name,
1465:     repo.name,
1466:     repo.description,
1467:     repo.language,
1468:     ...(Array.isArray(repo.topics) ? repo.topics : [])
1469:   ].join(" ").toLowerCase();
1470:   let score = repo.fork ? -10 : 12;
1471:   if (!repo.archived) score += 6;
1472:   if (repo.language) score += 4;
1473:   if (repo.name && !/\.github\.io$/i.test(repo.name)) score += 3;
1474:   tokens.forEach((token) => {
1475:     if (haystack.includes(token)) score += 22;
1476:   });
1477:   if (/\\b(vco|oscillator|pwm|analog|mixed|pcb|firmware|stm32|fpga|asic|verilog|embedded|signal|design)\\b/
i.test(haystack)) {
1478:     score += 12;
1479:   }
1480:   return score;
1481: }
```

fetchGitHubProfileSource (server.mjs, lines 1483-1543)

fetchGitHubProfileSource loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1483-1543 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references all, fetchGitHubJson, isArray, wantsGitHubCode, sort, scoreGitHubRepo, slice, fetchGitHubRepositorySource, trim, push, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1536: return {.

Important local values include owner at line 1484; repoList at line 1489; includeCode at line 1490; selectedRepos at line 1491; repoBlocks at line 1496; repoSource at line 1500; lines at line 1517. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1483: async function fetchGitHubProfileSource(source = {}, parsed = {}) {
1484:   const owner = parsed.owner;
1485:   const [profile, repos] = await Promise.all([
1486:     fetchGitHubJson(`https://api.github.com/users/${encodeURIComponent(owner)}`),
1487:     fetchGitHubJson(`https://api.github.com/users/${encodeURIComponent(owner)}/repos?per_page=30&sort=updated`)
1488:   ]);
1489:   const repoList = Array.isArray(repos) ? repos : [];
1490:   const includeCode = wantsGitHubCode(source.question || "");
1491:   const selectedRepos = includeCode
1492:     ? [...repoList]
1493:     .sort((a, b) => scoreGitHubRepo(b, source.question || "") - scoreGitHubRepo(a, source.question || ""))
1494:     .slice(0, 3)
1495:     : [];
1496:   const repoBlocks = [];
1497:
1498:   for (const repo of selectedRepos) {
1499:     if (!repo?.name || !repo?.html_url) continue;
1500:     const repoSource = await fetchGitHubRepositorySource({
1501:       ...source,
1502:       context: `Public GitHub repository selected from ${owner}'s profile`,
```

```

1503:     label: repo.full_name,
1504:     maxCodeFiles: 3,
1505:     maxFileTextLength: 3600,
1506:     maxRepositoryTextLength: 9000,
1507:     url: repo.html_url
1508:   }, {
1509:     owner,
1510:     repo: repo.name,
1511:     type: "repo",
1512:     branch: repo.default_branch
1513:   });
1514:   if (repoSource?.text?.trim()) repoBlocks.push(repoSource.text.trim());
1515: }
1516:
1517: const lines = [
1518:   `GitHub public profile: ${owner}`,
1519:   profile?.name ? `Name: ${profile.name}` : "",
1520:   profile?.bio ? `Bio: ${profile.bio}` : "",
1521:   profile?.html_url ? `Profile URL: ${profile.html_url}` : source.url,
1522:   "",
1523:   "Public repositories visible from the profile:",
1524:   ...repoList.slice(0, 20).map((repo) => [
1525:     `- ${repo.full_name}`,
1526:     repo.description ? `: ${repo.description}` : "",
1527:     repo.language ? ` | Language: ${repo.language}` : "",
1528:     repo.html_url ? ` | URL: ${repo.html_url}` : "",
1529:     repo.updated_at ? ` | Updated: ${repo.updated_at}` : ""
1530:   ]).join("")),
1531:   includeCode && selectedRepos.length ? "" : "",
1532:   includeCode && selectedRepos.length ? "Selected public repositories inspected for source code:" : "",
1533:   ...selectedRepos.map((repo) => `-${repo.full_name}${repo.language ? ` | Language: ${repo.language}` : ""}${repo.description ? ` | ${repo.description}` : ""}`),
1534:   ...repoBlocks.flatMap((block) => ["", "---", block])
1535: ].filter(Boolean);
1536: return {
1537:   context: source.context || "Public GitHub profile",
1538:   title: source.title || source.label || `GitHub profile: ${owner}`,
1539:   type: includeCode ? "public GitHub profile and selected repository code" : "public GitHub profile",
1540:   url: source.url,
1541:   text: clampText(lines.join("\n"), includeCode ? 30000 : 9000)
1542: };
1543: }

```

fetchGitHubRepositorySource (server.mjs, lines 1545-1612)

fetchGitHubRepositorySource loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1545-1612 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetchGitHubJson, wantsGitHubCode, max, min, isFinite, filter, fetchLimitedText, trim, push, clampText, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1569: return {}; line 1605: return {}.

Important local values include metadata at line 1547; branch at line 1548; question at line 1549; requestedMaxCodeFiles at line 1550; maxCodeFiles at line 1551; requestedFileTextLength at line 1552; maxFileTextLength at line 1553; requestedRepoTextLength at line 1554; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1545: async function fetchGitHubRepositorySource(source = {}, parsed = {}) {
1546:   const { owner, repo } = parsed;
1547:   const metadata = await
fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}`);
1548:   const branch = parsed.branch || metadata?.default_branch || "main";
1549:   const question = source.question || "";
1550:   const requestedMaxCodeFiles = Number(source.maxCodeFiles || (wantsGitHubCode(question) ? 6 : 3));
1551:   const maxCodeFiles = Math.max(1, Math.min(8, Number.isFinite(requestedMaxCodeFiles) ?
requestedMaxCodeFiles : 3));
1552:   const requestedFileTextLength = Number(source.maxFileTextLength || (wantsGitHubCode(question) ? 7000 :
3000));
1553:   const maxFileTextLength = Math.max(1000, Math.min(10000, Number.isFinite(requestedFileTextLength) ?
requestedFileTextLength : 3000));
1554:   const requestedRepoTextLength = Number(source.maxRepositoryTextLength || (wantsGitHubCode(question) ?
22000 : 14000));
1555:   const maxRepositoryTextLength = Math.max(5000, Math.min(26000, Number.isFinite(requestedRepoTextLength) ?
requestedRepoTextLength : 14000));
1556:   const lines = [
1557:     `GitHub repository: ${owner}/${repo}`,
1558:     metadata?.description ? `Description: ${metadata.description}` : "",
1559:     metadata?.language ? `Primary language: ${metadata.language}` : "",
1560:     metadata?.html_url ? `Repository URL: ${metadata.html_url}` : source.url,
1561:     metadata?.default_branch ? `Default branch: ${metadata.default_branch}` : "",
1562:     ""

```

```

1563:   ].filter(Boolean);
1564:
1565:   if (parsed.type === "file" && parsed.filePath) {
1566:     const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${parsed.filePath}`;
1567:     const text = await fetchLimitedText(rawUrl, 14000);
1568:     if (text.trim()) lines.push(`Source file: ${parsed.filePath}`, text.trim());
1569:     return {
1570:       context: source.context || "Public GitHub source file",
1571:       title: source.title || source.label || `${owner}/${repo}/${parsed.filePath}`,
1572:       type: "public GitHub source file",
1573:       url: source.url,
1574:       text: clampText(lines.join("\n"), 14000)
1575:     };
1576:   }
1577:
1578:   const tree = await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/git/trees/${encodeURIComponent(branch)}?recursive=1`);
1579:   const files = Array.isArray(tree?.tree)
1580:     ? tree.tree
1581:     : [];
1582:   .filter((item) => item.type === "blob" && item.path && githubTextFilePattern.test(item.path) &&
1583:     !githubSkipFilePattern.test(item.path))
1584:   .sort((a, b) => scoreGitHubFile(b.path, question) - scoreGitHubFile(a.path, question))
1585:   : [];
1586:
1587:   const readmePath = files.find((file) => /^(^|\/)readme\.md$/i.test(file.path)?.path || "README.md");
1588:   const readmeText = await
1589:   fetchLimitedText(`https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${readmePath}`, 9000).catch(() =>
1590:   "");
1591:   if (readmeText.trim()) lines.push(`README excerpt from ${readmePath}:`, readmeText.trim(), "");
1592:
1593:   if (files.length) {
1594:     lines.push("Repository text/code file list:", ...files.slice(0, 30).map((file) => `  ${file.path}`),
1595:     "");
1596:   }
1597:
1598:   const includeCode = wantsGitHubCode(question);
1599:   const selectedFiles = files
1600:     .filter((file) => !/readme\.md$/i.test(file.path))
1601:     .slice(0, maxCodeFiles);
1602:
1603:   for (const file of selectedFiles) {
1604:     const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${file.path}`;
1605:     const fileText = await fetchLimitedText(rawUrl, maxFileTextLength).catch(() => "");
1606:     if (!fileText.trim()) continue;
1607:     lines.push(`Source file: ${file.path}`, fileText.trim(), "");
1608:   }
1609:
1610:   return {
1611:     context: source.context || "Public GitHub repository",
1612:     title: source.title || source.label || `${owner}/${repo}`,
1613:     type: includeCode ? "public GitHub repository and code" : "public GitHub repository",
1614:     url: source.url,
1615:     text: clampText(lines.join("\n"), maxRepositoryTextLength)
1616:   };
1617: }

```

fetchGitHubSourceText (server.mjs, lines 1614-1619)

fetchGitHubSourceText loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1614-1619 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parseGitHubSourceUrl, fetchGitHubProfileSource, and fetchGitHubRepositorySource. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1616: if (!parsed) return null;; line 1617: if (parsed.type === "profile") return fetchGitHubProfileSource(source, parsed);; line 1618: return fetchGitHubRepositorySource(source, parsed);.

Important local values include parsed at line 1615. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1614: async function fetchGitHubSourceText(source = {}) {
1615:   const parsed = parseGitHubSourceUrl(source.url || "");
1616:   if (!parsed) return null;
1617:   if (parsed.type === "profile") return fetchGitHubProfileSource(source, parsed);
1618:   return fetchGitHubRepositorySource(source, parsed);
1619: }

```

readLocalSourceText (server.mjs, lines 1621-1639)

readLocalSourceText loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1621-1639 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URL, Set, has, toLowerCase, replace, includes, extname, readFile, and resolveInsideRoot. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 1626: return ""; line 1629: if (!localHosts.has(parsed.hostname.toLowerCase())) return ""; line 1631: if (!pathname || pathname.includes("..")) return ""; line 1633: if (![".txt", ".md", ".json", ".csv", ".log", ".xml", ".js", ".mjs", ".css", ".c", ".h", ".cpp", ".hpp", ".py", ".v", ".sv", ".spice", ".cir", ".net", ".asc", ".sch", ".kicad_sch", ".kicad_pcb"].includes(ext)) return ""; There are 2 additional return statements in the function body.

Important local values include localHosts at line 1628; pathname at line 1630; ext at line 1632. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1621: async function readLocalSourceText(url) {
1622:   let parsed;
1623:   try {
1624:     parsed = new URL(url);
1625:   } catch {
1626:     return "";
1627:   }
1628:   const localHosts = new Set(["localhost", "127.0.0.1"]);
1629:   if (!localHosts.has(parsed.hostname.toLowerCase())) return "";
1630:   const pathname = decodeURIComponent(parsed.pathname.replace(/^\/+/, ""));
1631:   if (!pathname || pathname.includes("..")) return "";
1632:   const ext = path.extname(pathname).toLowerCase();
1633:   if (![".txt", ".md", ".json", ".csv", ".log", ".xml", ".js", ".mjs", ".css", ".c", ".h", ".cpp",
".hpp", ".py", ".v", ".sv", ".spice", ".cir", ".net", ".asc", ".sch", ".kicad_sch", ".kicad_pcb"].includes(ext))
return "";
1634:   try {
1635:     return await readFile(resolveInsideRoot(pathname), "utf8");
1636:   } catch {
1637:     return "";
1638:   }
1639: }
```

fetchSourceText (server.mjs, lines 1641-1679)

fetchSourceText loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1641-1679 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sourceLooksTextual, sourceUrlAllowed, fetchGitHubSourceText, trim, readLocalSourceText, clampText, replace, fetch, get, text, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 8 return statement(s). The most important visible returns are: line 1643: if (!url || !sourceLooksTextual(source) || !sourceUrlAllowed(url)) return null;; line 1646: if (githubText?.text?.trim()) return githubText;; line 1650: return {}; line 1663: if (!response.ok) return null;. There are 4 additional return statements in the function body.

Important local values include url at line 1642; githubText at line 1645; localText at line 1648; response at line 1660; contentType at line 1664; rawText at line 1666; text at line 1667. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1641: async function fetchSourceText(source = {}) {
1642:   const url = String(source.url || "");
1643:   if (!url || !sourceLooksTextual(source) || !sourceUrlAllowed(url)) return null;
1644:   if (/github\.com|raw\.githubusercontent\.com/i.test(url)) {
1645:     const githubText = await fetchGitHubSourceText(source);
1646:     if (githubText?.text?.trim()) return githubText;
1647:   }
1648:   const localText = await readLocalSourceText(url);
1649:   if (localText.trim()) {
1650:     return {
1651:       context: source.context || "",
1652:       title: source.title || source.label || url,
1653:       type: source.type || source.kind || "text source",
1654:       url,
1655:       text: clampText(localText.replace(/\s+\n/g, "\n").trim(), 9000)
1656:     };
1657:   }
1658:
1659:   try {
1660:     const response = await fetch(url, {
```

```

1661:     headers: { "Accept": "text/plain,text/markdown,text/html,application/json;q=0.8,*/*;q=0.1" }
1662:   });
1663:   if (!response.ok) return null;
1664:   const contentType = response.headers.get("Content-Type") || "";
1665:   if (!/text|json|xml|html|markdown/i.test(contentType)) return null;
1666:   const rawText = clampText(await response.text(), 12000);
1667:   const text = /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;
1668:   if (!text.trim()) return null;
1669:   return {
1670:     context: source.context || "",
1671:     title: source.title || source.label || url,
1672:     type: source.type || source.kind || "public source",
1673:     url,
1674:     text: clampText(text.trim(), 9000)
1675:   };
1676: } catch {
1677:   return null;
1678: }
1679: }

```

enrichPortfolioContext (server.mjs, lines 1681-1692)

This function expands the AI context with portfolio content and public source text when allowed. It can read local public files or fetch public GitHub/link data so the assistant has richer material for portfolio-aware answers.

The AI should not give canned answers. It needs enough context to distinguish a general electronics question from a project-specific question and answer with relevant evidence.

It calls or references `isArray`, `slice`, `map`, `all`, and `filter`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1687: `return {`.

Important local values include `question` at line 1682; `sources` at line 1683; `sourceExcerpts` at line 1686. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1681: async function enrichPortfolioContext(context = {}) {
1682:   const question = String(context.question || "");
1683:   const sources = Array.isArray(context.sourceFetches)
1684:     ? context.sourceFetches.slice(0, 10).map((source) => ({ ...source, question: source.question ||
question }))
1685:     : [];
1686:   const sourceExcerpts = (await Promise.all(sources.map(fetchSourceText))).filter(Boolean);
1687:   return {
1688:     ...context,
1689:     sourceExcerpts,
1690:     sourceFetchPolicy: "Fetched excerpts are limited to safe text-like same-site, GitHub/raw GitHub,
LinkedIn, and public portfolio URLs. GitHub repository links can be expanded into repository metadata, README
text, and selected public source files when a question asks for code. PDFs/images are represented by captions,
metadata, filenames, and companion text files when available."
1691:   };
1692: }

```

cleanConversationHistory (server.mjs, lines 1694-1703)

`cleanConversationHistory` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1694-1703 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `isArray`, `slice`, `map`, `clampText`, `trim`, and `filter`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1695: `if (!Array.isArray(value)) return [];`; line 1696: `return value`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1694: function cleanConversationHistory(value) {
1695:   if (!Array.isArray(value)) return [];
1696:   return value
1697:     .slice(-8)
1698:     .map((item) => ({
1699:       role: item?.role === "assistant" ? "assistant" : "user",
1700:       content: clampText(item?.content, 1400).trim()
1701:     }))
1702:     .filter((item) => item.content);
1703: }

```


extractOpenAiText (server.mjs, lines 1705-1717)

extractOpenAiText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1705-1717 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, isArray, forEach, push, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1706: if (typeof data?.output_text === "string" && data.output_text.trim()) return data.output_text.trim(); line 1716: return chunks.join("\n\n").trim();.

Important local values include output at line 1707; chunks at line 1708. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1705: function extractOpenAiText(data) {
1706:   if (typeof data?.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
1707:   const output = Array.isArray(data?.output) ? data.output : [];
1708:   const chunks = [];
1709:
1710:   output.forEach((item) => {
1711:     (item.content || []).forEach((content) => {
1712:       if (typeof content.text === "string" && content.text.trim()) chunks.push(content.text.trim());
1713:     });
1714:   });
1715:
1716:   return chunks.join("\n\n").trim();
1717: }
```

extractOllamaText (server.mjs, lines 1719-1723)

extractOllamaText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1719-1723 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1720: if (typeof data.message?.content === "string") return data.message.content.trim(); line 1721: if (typeof data.response === "string") return data.response.trim(); line 1722: return "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1719: function extractOllamaText(data = {}) {
1720:   if (typeof data.message?.content === "string") return data.message.content.trim();
1721:   if (typeof data.response === "string") return data.response.trim();
1722:   return "";
1723: }
```

callOllamaPortfolioAi (server.mjs, lines 1725-1773)

callOllamaPortfolioAi part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1725-1773 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references AbortController, abort, fetch, replace, stringify, clampText, join, json, and extractOllamaText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 1762: return { ok: false, error: data?.error || `Ollama returned HTTP \${response.status}.` }; line 1765: return answer; line 1767: : { ok: false, error: "Ollama did not return an answer." }; line 1769: return { ok: false, error: error?.name === "AbortError" ? "Ollama timed out." : "Ollama is not reachable on this machine." };.

Important local values include host at line 1726; model at line 1727; controller at line 1728; timeout at line 1729; response at line 1731; data at line 1760; answer at line 1764. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1725: async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch }) {
1726:   const host = process.env.OLLAMA_HOST || "http://127.0.0.1:11434";
1727:   const model = process.env.OLLAMA_MODEL || "llama3.2";
1728:   const controller = new AbortController();
1729:   const timeout = setTimeout(() => controller.abort(), Number(process.env.OLLAMA_TIMEOUT_MS || 45000));
1730:   try {
1731:     const response = await fetch(`${host.replace(/\/+$/, "")}/api/chat`, {
1732:       method: "POST",
1733:       headers: { "Content-Type": "application/json" },
1734:       signal: controller.signal,
1735:       body: JSON.stringify({
1736:         model,
1737:         stream: false,
1738:         messages: [
1739:           { role: "system", content: portfolioAiInstructions },
1740:           {
1741:             role: "user",
1742:             content: [
1743:               `Visitor question: ${question}`,
1744:               `Question intent: ${intent}`,
1745:               `Web/public lookup requested by browser: ${allowWebSearch ? "yes" : "no"}`,
1746:               "",
1747:               "Recent conversation JSON:",
1748:               clampText(JSON.stringify(conversation, null, 2), 8000),
1749:               "",
1750:               "Portfolio context JSON:",
1751:               clampText(JSON.stringify(context, null, 2), 18000)
1752:             ].join("\n")
1753:           }
1754:         ],
1755:         options: {
1756:           temperature: Number(process.env.OLLAMA_TEMPERATURE || 0.35)
1757:         }
1758:       })
1759:     });
1760:     const data = await response.json().catch(() => ({}));
1761:     if (!response.ok) {
1762:       return { ok: false, error: data?.error || `Ollama returned HTTP ${response.status}` };
1763:     }
1764:     const answer = extractOllamaText(data);
1765:     return answer
1766:       ? { ok: true, answer, model }
1767:       : { ok: false, error: "Ollama did not return an answer." };
1768:   } catch (error) {
1769:     return { ok: false, error: error?.name === "AbortError" ? "Ollama timed out." : "Ollama is not reachable on this machine." };
1770:   } finally {
1771:     clearTimeout(timeout);
1772:   }
1773: }
```

ruleBasedConversationAnswer (server.mjs, lines 1775-1786)

ruleBasedConversationAnswer part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1775-1786 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1778: return "You're welcome. I can keep helping with this portfolio, project evidence, resume links, or related electronics and embedded-systems questions."; line 1782: return "I am this portfolio's assistant. I can help visitors explore the saved engineering work, explain project details, summarize portfolio evidence, and answer related electronics, embedded, analog, digital, FPGA, ASI; line 1785: return "Hi. I can help you explore this portfolio, explain projects, summarize project evidence, open relevant sections, or answer related electronics and embedded-systems questions. You can ask about a specific project,.

Important local values include clean at line 1776. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1775: function ruleBasedConversationAnswer(question = "") {
1776:   const clean = String(question || "").toLowerCase();
1777:   if (/^b(thanks|thank you|appreciate it)\b/.test(clean)) {
1778:     return "You're welcome. I can keep helping with this portfolio, project evidence, resume links, or related electronics and embedded-systems questions.";
1779:   }
1780:
1781:   if (/^b(who are you|what are you)\b/.test(clean)) {
1782:     return "I am this portfolio's assistant. I can help visitors explore the saved engineering work,
```

```

explain project details, summarize portfolio evidence, and answer related electronics, embedded, analog,
digital, FPGA, ASIC, PCB, and firmware questions.";
1783: }
1784:
1785: return "Hi. I can help you explore this portfolio, explain projects, summarize project evidence, open
relevant sections, or answer related electronics and embedded-systems questions. You can ask about a specific
project, a tool like KiCad or STM32CubeIDE, or a general topic such as embedded systems, op amps, FPGA design,
ASICs, or PCB testing.";
1786: }

```

isLocalRequest (server.mjs, lines 1788-1791)

isLocalRequest part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1788-1791 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1790: return address === "127.0.0.1" || address === "::1" || address === "::ffff:127.0.0.1";.

Important local values include address at line 1789. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1788: function isLocalRequest(request) {
1789:   const address = request.socket.remoteAddress || "";
1790:   return address === "127.0.0.1" || address === "::1" || address === "::ffff:127.0.0.1";
1791: }

```

readJsonFile (server.mjs, lines 1793-1795)

readJsonFile loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1793-1795 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, and readFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1794: return JSON.parse(await readFile(filePath, "utf8"));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1793: async function readJsonFile(filePath) {
1794:   return JSON.parse(await readFile(filePath, "utf8"));
1795: }

```

readRequestJson (server.mjs, lines 1797-1810)

readRequestJson loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1797-1810 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Error, push, parse, concat, toString, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1809: return JSON.parse((Buffer.concat(chunks).toString("utf8") || "{}").replace(/^\uFEFF/, ""));.

Important local values include chunks at line 1798; size at line 1799. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1797: async function readRequestJson(request) {
1798:   const chunks = [];
1799:   let size = 0;

```

```

1800:
1801:   for await (const chunk of request) {
1802:     size += chunk.length;
1803:     if (size > 60 * 1024 * 1024) {
1804:       throw new Error("Request body is too large.");
1805:     }
1806:     chunks.push(chunk);
1807:   }
1808:
1809:   return JSON.parse((Buffer.concat(chunks).toString("utf8") || "{}").replace(/^\uFEFF/, ""));
1810: }

```

safeSegment (server.mjs, lines 1812-1819)

safeSegment validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1812-1819 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, trim, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1818: return segment || fallback;.

Important local values include segment at line 1813. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1812: function safeSegment(value, fallback = "item") {
1813:   const segment = String(value || fallback)
1814:     .toLowerCase()
1815:     .trim()
1816:     .replace(/^[a-z0-9._-]+/g, "-")
1817:     .replace(/^-+|-+$/g, "");
1818:   return segment || fallback;
1819: }

```

safeFileName (server.mjs, lines 1821-1826)

safeFileName validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1821-1826 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, safeSegment, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1825: return ext ? `\${name}.\${ext}` : name;.

Important local values include parsed at line 1822; name at line 1823; ext at line 1824. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1821: function safeFileName(value) {
1822:   const parsed = path.parse(String(value || "file"));
1823:   const name = safeSegment(parsed.name, "file");
1824:   const ext = safeSegment(parsed.ext.replace(".", ""), "");
1825:   return ext ? `${name}.${ext}` : name;
1826: }

```

resolveInsideRoot (server.mjs, lines 1828-1834)

resolveInsideRoot part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1828-1834 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, join, startsWith, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1833: return target;.

Important local values include target at line 1829. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1828: function resolveInsideRoot(...segments) {
1829:   const target = path.normalize(path.join(root, ...segments));
1830:   if (!target.startsWith(root)) {
1831:     throw new Error("Resolved path is outside the workspace.");
1832:   }
1833:   return target;
1834: }
```

resolveInsidePortfolioRoot (server.mjs, lines 1836-1843)

resolveInsidePortfolioRoot part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1836-1843 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resolve, relative, startsWith, isAbsolute, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1842: return target;.

Important local values include target at line 1837; relative at line 1838. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1836: function resolveInsidePortfolioRoot(...segments) {
1837:   const target = path.resolve(portfolioRoot, ...segments);
1838:   const relative = path.relative(portfolioRoot, target);
1839:   if (relative.startsWith(".") || path.isAbsolute(relative)) {
1840:     throw new Error("Resolved path is outside the portfolio workspace.");
1841:   }
1842:   return target;
1843: }
```

resolveInsideCompileRoot (server.mjs, lines 1845-1852)

resolveInsideCompileRoot part of the compile code language/tool/build/run flow. In this file, it appears around lines 1845-1852 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resolve, relative, startsWith, isAbsolute, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1851: return target;.

Important local values include target at line 1846; relative at line 1847. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1845: function resolveInsideCompileRoot(...segments) {
1846:   const target = path.resolve(compileRoot, ...segments);
1847:   const relative = path.relative(compileRoot, target);
1848:   if (relative.startsWith(".") || path.isAbsolute(relative)) {
1849:     throw new Error("Resolved path is outside the compile workspace.");
1850:   }
1851:   return target;
1852: }
```

samePath (server.mjs, lines 1854-1856)

samePath part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1854-1856 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `resolve`, and `toLowerCase`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1855: `return path.resolve(left).toLowerCase() === path.resolve(right).toLowerCase();`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1854: function samePath(left, right) {  
1855:   return path.resolve(left).toLowerCase() === path.resolve(right).toLowerCase();  
1856: }
```

pathExists (server.mjs, lines 1858-1865)

`pathExists` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1858-1865 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `fsAccess`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1861: `return true;`; line 1863: `return false;`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1858: async function pathExists(filePath) {  
1859:   try {  
1860:     await fsAccess(filePath);  
1861:     return true;  
1862:   } catch {  
1863:     return false;  
1864:   }  
1865: }
```

publishAccessError (server.mjs, lines 1867-1877)

`publishAccessError` part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1867-1877 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `Error`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1876: `return error;`.

Important local values include `error` at line 1868. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1867: function publishAccessError(message, details = "", extra = {}) {  
1868:   const error = new Error(message);  
1869:   error.code = "PUBLISH_AUTHORIZATION_REQUIRED";  
1870:   error.details = details || publishAuthorizationHelp;  
1871:   error.publishAccess = {  
1872:     authorizationRequired: true,  
1873:     help: publishAuthorizationHelp,  
1874:     ...extra  
1875:   };  
1876:   return error;  
1877: }
```

gitFailureText (server.mjs, lines 1879-1885)

`gitFailureText` part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1879-1885 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, join, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1880: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1879: function gitFailureText(error) {
1880:   return [
1881:     error?.stderr,
1882:     error?.stdout,
1883:     error?.message
1884:   ].filter(Boolean).join("\n").trim();
1885: }
```

remoteUrlForDisplay (server.mjs, lines 1887-1891)

remoteUrlForDisplay part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1887-1891 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1888: return String(remoteUrl || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1887: function remoteUrlForDisplay(remoteUrl = "") {
1888:   return String(remoteUrl || "")
1889:     .replace(/^https:\\\\\/[^\:\/]+):[^\:\/]+@\/i, "https://$1:***@")
1890:     .replace(/^https:\\\\\/[^\:\/]+@\/i, "https://");
1891: }
```

parseGitHubRemote (server.mjs, lines 1893-1907)

parseGitHubRemote part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1893-1907 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, match, replace, URL, and split. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 1896: if (sshMatch) return { owner: sshMatch[1], repo: sshMatch[2].replace(/\.git\$/i, "") }; line 1900: if (!/github\.com\$/i.test(parsed.hostname)) return null; line 1902: if (parts.length < 2) return null; line 1903: return { owner: parts[0], repo: parts[1].replace(/\.git\$/i, "") };. There are 1 additional return statements in the function body.

Important local values include trimmed at line 1894; sshMatch at line 1895; parsed at line 1899; parts at line 1901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1893: function parseGitHubRemote(remoteUrl = "") {
1894:   const trimmed = String(remoteUrl || "").trim();
1895:   const sshMatch = trimmed.match(/^git@github\.com:([^\:\/]+)\.?(?:\.git)?$/i);
1896:   if (sshMatch) return { owner: sshMatch[1], repo: sshMatch[2].replace(/\.git$/i, "") };
1897:
1898:   try {
1899:     const parsed = new URL(trimmed.replace(/^(git|https):\/\//, ""));
1900:     if (!/github\.com$/i.test(parsed.hostname)) return null;
1901:     const parts = parsed.pathname.replace(/^(\/|\/)/, "").split("/");
1902:     if (parts.length < 2) return null;
1903:     return { owner: parts[0], repo: parts[1].replace(/\.git$/i, "") };
1904:   } catch {
1905:     return null;
1906:   }
1907: }
```

validatePublishRemoteUrl (server.mjs, lines 1909-1927)

validatePublishRemoteUrl part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1909-1927 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, URL, toLowerCase, Error, replace, split, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1911: if (!remoteUrl) return ""; line 1912: if (/^git@github\.com:[A-Za-z0-9_.-]+\V[A-Za-z0-9_.-]+(?:\.git)?\$/i.test(remoteUrl)) return remoteUrl; line 1922: return `https://github.com/\${parts[0]}/\${parts[1].replace(/\./i, "")}.git`;

Important local values include remoteUrl at line 1910; parsed at line 1914; parts at line 1918. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1909: function validatePublishRemoteUrl(value = "") {
1910:   const remoteUrl = String(value || "").trim();
1911:   if (!remoteUrl) return "";
1912:   if (/^git@github\.com:[A-Za-z0-9_.-]+\V[A-Za-z0-9_.-]+(?:\.git)?$/i.test(remoteUrl)) return remoteUrl;
1913:   try {
1914:     const parsed = new URL(remoteUrl);
1915:     if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {
1916:       throw new Error("Only GitHub HTTPS repository URLs are supported by the guided setup.");
1917:     }
1918:     const parts = parsed.pathname.replace(/^\/+/, "").split("/");
1919:     if (parts.length < 2 || !parts[0] || !parts[1]) {
1920:       throw new Error("GitHub repository URL must include owner and repository name.");
1921:     }
1922:     return `https://github.com/${parts[0]}/${parts[1].replace(/\./i, "")}.git`;
1923:   } catch (error) {
1924:     if (error.message?.includes("Only GitHub") || error.message?.includes("owner")) throw error;
1925:     throw new Error("Enter a GitHub repository URL such as
https://github.com/USERNAME/USERNAME.github.io.git.");
1926:   }
1927: }
```

validateCustomDomain (server.mjs, lines 1929-1936)

validateCustomDomain validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1929-1936 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1931: if (!domain) return ""; line 1935: return domain;

Important local values include domain at line 1930. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1929: function validateCustomDomain(value = "") {
1930:   const domain = String(value || "").trim().toLowerCase();
1931:   if (!domain) return "";
1932:   if (!/^(?:[a-z0-9]{0,61}[a-z0-9])?\.\.[a-z]{2,63}$/i.test(domain)) {
1933:     throw new Error("Custom domain must look like example.com or portfolio.example.com.");
1934:   }
1935:   return domain;
1936: }
```

compareVersions (server.mjs, lines 1938-1947)

compareVersions part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1938-1947 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1944: if (delta !== 0) return delta;; line 1946: return 0;.

Important local values include leftParts at line 1939; rightParts at line 1940; length at line 1941; delta at line 1943. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1938: function compareVersions(left = "", right = "") {
1939:   const leftParts = String(left || "0").split(/[. -]/).map((part) => Number.parseInt(part, 10) || 0);
1940:   const rightParts = String(right || "0").split(/[. -]/).map((part) => Number.parseInt(part, 10) || 0);
1941:   const length = Math.max(leftParts.length, rightParts.length);
1942:   for (let index = 0; index < length; index += 1) {
1943:     const delta = (leftParts[index] || 0) - (rightParts[index] || 0);
1944:     if (delta !== 0) return delta;
1945:   }
1946:   return 0;
1947: }
```

bumpPublishedAssetVersions (server.mjs, lines 1949-1965)

bumpPublishedAssetVersions part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1949-1965 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join, readFile, now, toString, replace, and writeFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1955: return false;; line 1962: if (next === html) return false;; line 1964: return true;.

Important local values include html at line 1950; indexPath at line 1951; version at line 1957; next at line 1958. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1949: async function bumpPublishedAssetVersions(baseRoot = portfolioRoot) {
1950:   let html = "";
1951:   const indexPath = path.join(baseRoot, "index.html");
1952:   try {
1953:     html = await readFile(indexPath, "utf8");
1954:   } catch {
1955:     return false;
1956:   }
1957:   const version = Date.now().toString();
1958:   const next = html
1959:     .replace(/(href="styles\.css)(?:\?v=[^"]*)?"/g, ` $1?v=${version}`)
1960:     .replace(/(src="script\.js)(?:\?v=[^"]*)?"/g, ` $1?v=${version}`)
1961:     .replace(/(src="electronics-search\.js)(?:\?v=[^"]*)?"/g, ` $1?v=${version}`);
1962:   if (next === html) return false;
1963:   await writeFile(indexPath, next, "utf8");
1964:   return true;
1965: }
```

workspaceHasCompatibleSiteFiles (server.mjs, lines 1967-1976)

workspaceHasCompatibleSiteFiles part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1967-1976 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references readFile, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1972: return false;; line 1975: return true;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1967: async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot) {
1968:   for (const fileName of ["index.html", "styles.css", "script.js"]) {
1969:     try {
1970:       await readFile(path.join(baseRoot, fileName));
1971:     } catch {
1972:       return false;
1973:     }
1974:   }
1975:   return true;
1976: }
```

```

1974:   }
1975:   return true;
1976: }

```

getPublishTargetInfo (server.mjs, lines 1978-2044)

getPublishTargetInfo part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1978-2044 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references samePath, workspaceHasCompatibleSiteFiles, runPublishGit, trim, remoteUrlForDisplay, parseGitHubRemote, readFile, resolveInsidePortfolioRoot, readPublishAuthCache, publishAuthCachelsFresh, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1995: return info;; line 2043: return info;.

Important local values include info at line 1979; parsedRemote at line 2010; accessShape at line 2019; cached at line 2024. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1978: async function getPublishTargetInfo() {
1979:   const info = {
1980:     workspace: root,
1981:     portfolioWorkspace: portfolioRoot,
1982:     separatedWorkspace: !samePath(root, portfolioRoot),
1983:     defaultRepository: defaultSiteRepository,
1984:     gitBacked: false,
1985:     compatible: await workspaceHasCompatibleSiteFiles(),
1986:     remote: "",
1987:     branch: "",
1988:     repository: "",
1989:     customDomain: ""
1990:   };
1991:
1992:   try {
1993:     info.gitBacked = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim() ===
"true";
1994:   } catch {
1995:     return info;
1996:   }
1997:
1998:   try {
1999:     info.remote = remoteUrlForDisplay((await runPublishGit(["remote", "get-url",
"origin"])).stdout.trim());
2000:   } catch {
2001:     info.remote = "";
2002:   }
2003:
2004:   try {
2005:     info.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2006:   } catch {
2007:     info.branch = "";
2008:   }
2009:
2010:   const parsedRemote = parseGitHubRemote(info.remote);
2011:   info.repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : info.remote;
2012:
2013:   try {
2014:     info.customDomain = (await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8")).trim();
2015:   } catch {
2016:     info.customDomain = "";
2017:   }
2018:
2019:   const accessShape = {
2020:     branch: info.branch,
2021:     remote: info.remote,
2022:     repository: info.repository
2023:   };
2024:   const cached = await readPublishAuthCache();
2025:   if (publishAuthCacheIsFresh(cached, accessShape)) {
2026:     info.authorizationChecked = true;
2027:     info.authorizationCached = true;
2028:     info.checkedAt = cached.checkedAt;
2029:     info.expiresAt = publishAuthCacheExpiresAt(cached);
2030:     info.extendedTrust = Boolean(cached.extendedTrust);
2031:     info.successCountLastWeek = cached.successCountLastWeek || 0;
2032:     info.trustDays = cached.trustDays || (cached.extendedTrust ? 30 : 1);
2033:   } else {
2034:     info.authorizationChecked = false;
2035:     info.authorizationCached = false;
2036:     info.checkedAt = "";
2037:     info.expiresAt = "";
2038:     info.extendedTrust = false;
2039:     info.successCountLastWeek = cached?.successCountLastWeek || 0;

```

```

2040:     info.trustDays = 0;
2041:   }
2042:
2043:   return info;
2044: }

```

runGit (server.mjs, lines 2046-2064)

runGit part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2046-2064 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references execFileAsync, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2056: return { ...result, git: candidate };

Important local values include lastError at line 2047; result at line 2050. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2046: async function runGit(args, options = {}) {
2047:   let lastError = null;
2048:   for (const candidate of gitCandidates) {
2049:     try {
2050:       const result = await execFileAsync(candidate, args, {
2051:         cwd: options.cwd || root,
2052:         maxBuffer: options.maxBuffer || 10 * 1024 * 1024,
2053:         timeout: options.timeout || 0,
2054:         windowsHide: options.windowsHide !== false
2055:       });
2056:       return { ...result, git: candidate };
2057:     } catch (error) {
2058:       lastError = error;
2059:       if (error.code === "ENOENT") continue;
2060:       throw error;
2061:     }
2062:   }
2063:   throw lastError || new Error("Git executable was not found.");
2064: }

```

runPublishGit (server.mjs, lines 2066-2068)

runPublishGit part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2066-2068 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2067: return runGit(args, { ...options, cwd: options.cwd || portfolioRoot });

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2066: async function runPublishGit(args, options = {}) {
2067:   return runGit(args, { ...options, cwd: options.cwd || portfolioRoot });
2068: }

```

runOptionalCommand (server.mjs, lines 2070-2087)

runOptionalCommand part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2070-2087 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references execFileAsync. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2078: return { ok: true, stdout: result.stdout || "", stderr: result.stderr || "" };; line 2080: return {.

Important local values include result at line 2072. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2070: async function runOptionalCommand(command, args = [], options = {}) {
2071:   try {
2072:     const result = await execFileAsync(command, args, {
2073:       cwd: options.cwd || root,
2074:       maxBuffer: options.maxBuffer || 2 * 1024 * 1024,
2075:       timeout: options.timeout || 15000,
2076:       windowsHide: options.windowsHide !== false
2077:     });
2078:     return { ok: true, stdout: result.stdout || "", stderr: result.stderr || "" };
2079:   } catch (error) {
2080:     return {
2081:       ok: false,
2082:       stdout: error.stdout || "",
2083:       stderr: error.stderr || "",
2084:       error: error.message || String(error)
2085:     };
2086:   }
2087: }
```

getGitStatus (server.mjs, lines 2089-2136)

getGitStatus part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2089-2136 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runGit, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2113: return {.

Important local values include git at line 2090; gitResult at line 2092; credentialManager at line 2097; gcm at line 2099; gcmResult at line 2101. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2089: async function getGitStatus() {
2090:   let git = { ok: false, stdout: "", stderr: "", error: "Git for Windows was not found." };
2091:   try {
2092:     const gitResult = await runGit(["--version"]);
2093:     git = { ok: true, stdout: gitResult.stdout || "", stderr: gitResult.stderr || "" };
2094:   } catch (error) {
2095:     git = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message ||
"Git for Windows was not found." };
2096:   }
2097:   let credentialManager = { ok: false, version: "", error: "Git Credential Manager was not found." };
2098:   if (git.ok) {
2099:     let gcm = { ok: false, stdout: "", stderr: "", error: "Git Credential Manager was not found." };
2100:     try {
2101:       const gcmResult = await runGit(["credential-manager", "--version"]);
2102:       gcm = { ok: true, stdout: gcmResult.stdout || "", stderr: gcmResult.stderr || "" };
2103:     } catch (error) {
2104:       gcm = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message ||
"Git Credential Manager was not found." };
2105:     }
2106:     credentialManager = {
2107:       ok: gcm.ok,
2108:       version: (gcm.stdout || gcm.stderr || "").trim(),
2109:       error: gcm.ok ? "" : (gcm.stderr || gcm.error || "Git Credential Manager was not found.")
2110:     };
2111:   }
2112:
2113:   return {
2114:     git: {
2115:       ok: git.ok,
2116:       version: (git.stdout || git.stderr || "").trim(),
2117:       error: git.ok ? "" : (git.stderr || git.error || "Git for Windows was not found.")
2118:     },
2119:     credentialManager,
2120:     nodeRuntime: {
2121:       ok: true,
2122:       version: process.versions.node,
2123:       bundled: true,
2124:       note: "Bundled inside the desktop app. No manual Node.js install is required."
2125:     },
2126:     npnm: {
2127:       ok: true,
```

```

2128:         required: false,
2129:         note: "pnpm is only needed by developers building the installer from source."
2130:     },
2131:     app: {
2132:         version: process.env.OMB_APP_VERSION || packageJson.version || "0.0.0",
2133:         desktopApp: process.env.OMB_DESKTOP_APP === "1"
2134:     }
2135: };
2136: }

```

publishPathIsStageable (server.mjs, lines 2138-2152)

publishPathIsStageable part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2138-2152 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fsAccess, resolveInsidePortfolioRoot, and runPublishGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2141: return true;; line 2148: return true;; line 2150: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2138: async function publishPathIsStageable(relativePath) {
2139:   try {
2140:     await fsAccess(resolveInsidePortfolioRoot(relativePath));
2141:     return true;
2142:   } catch {
2143:     // Missing files can still be stageable if Git already tracks them and they were deleted.
2144:   }
2145:
2146:   try {
2147:     await runPublishGit(["ls-files", "--error-unmatch", "--", relativePath]);
2148:     return true;
2149:   } catch {
2150:     return false;
2151:   }
2152: }

```

stageablePublishPaths (server.mjs, lines 2154-2160)

stageablePublishPaths part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2154-2160 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references publishPathIsStageable, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2159: return stageable;.

Important local values include stageable at line 2155. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2154: async function stageablePublishPaths(pathsToCheck) {
2155:   const stageable = [];
2156:   for (const relativePath of pathsToCheck) {
2157:     if (await publishPathIsStageable(relativePath)) stageable.push(relativePath);
2158:   }
2159:   return stageable;
2160: }

```

runGitWithInput (server.mjs, lines 2162-2204)

runGitWithInput part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2162-2204 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references spawn, on, toString, once, resolve, Error, reject, and end. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2166: `return await new Promise((resolve, reject) => {`; line 2185: `return;`.

Important local values include `lastError` at line 2163; `child` at line 2167; `stdout` at line 2172; `stderr` at line 2173; `error` at line 2187. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2162: async function runGitWithInput(args, input, options = {}) {
2163:   let lastError = null;
2164:   for (const candidate of gitCandidates) {
2165:     try {
2166:       return await new Promise((resolve, reject) => {
2167:         const child = spawn(candidate, args, {
2168:           cwd: options.cwd || portfolioRoot,
2169:           windowsHide: true,
2170:           stdio: ["pipe", "pipe", "pipe"]
2171:         });
2172:         let stdout = "";
2173:         let stderr = "";
2174:
2175:         child.stdout.on("data", (chunk) => {
2176:           stdout += chunk.toString();
2177:         });
2178:         child.stderr.on("data", (chunk) => {
2179:           stderr += chunk.toString();
2180:         });
2181:         child.once("error", reject);
2182:         child.once("close", (code) => {
2183:           if (code === 0) {
2184:             resolve({ stdout, stderr, git: candidate });
2185:             return;
2186:           }
2187:           const error = new Error(stderr || stdout || `Git exited with code ${code}.`);
2188:           error.stdout = stdout;
2189:           error.stderr = stderr;
2190:           error.code = code;
2191:           error.git = candidate;
2192:           reject(error);
2193:         });
2194:
2195:         child.stdin.end(input);
2196:       });
2197:     } catch (error) {
2198:       lastError = error;
2199:       if (error.code === "ENOENT") continue;
2200:       throw error;
2201:     }
2202:   }
2203:   throw lastError || new Error("Git executable was not found.");
2204: }
```

storeGitCredentials (server.mjs, lines 2206-2237)

`storeGitCredentials` persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2206-2237 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `trim`, `Error`, `URL`, `toLowerCase`, `replace`, `runPublishGit`, `join`, and `runGitWithInput`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2209: `if (!cleanUsername && !cleanPassword) return { stored: false };`; line 2236: `return { stored: true, username: cleanUsername };`.

Important local values include `cleanUsername` at line 2207; `cleanPassword` at line 2208; `parsed` at line 2214; `pathName` at line 2225; `credentialInput` at line 2227. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2206: async function storeGitCredentials(remoteUrl, username, password) {
2207:   const cleanUsername = String(username || "").trim();
2208:   const cleanPassword = String(password || "");
2209:   if (!cleanUsername && !cleanPassword) return { stored: false };
2210:   if (!cleanUsername || !cleanPassword) {
2211:     throw new Error("Both username and password/token are required when credentials are provided.");
2212:   }
2213:
2214:   let parsed = null;
2215:   try {
2216:     parsed = new URL(remoteUrl);
2217:   } catch {
2218:     throw new Error("Username and password/token credentials require a GitHub HTTPS repository URL.");
2219:   }
```

```

2220:
2221:   if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {
2222:     throw new Error("Username and password/token credentials require a GitHub HTTPS repository URL.");
2223:   }
2224:
2225:   const pathName = parsed.pathname.replace(/^\/+/, "");
2226:   await runPublishGit(["config", "--local", "credential.useHttpPath", "true"]);
2227:   const credentialInput = [
2228:     "protocol=https",
2229:     "host=github.com",
2230:     `path=${pathName}`,
2231:     `username=${cleanUsername}`,
2232:     `password=${cleanPassword}`,
2233:     ""
2234:   ].join("\n");
2235:   await runGitWithInput(["credential", "approve"], credentialInput, { cwd: portfolioRoot });
2236:   return { stored: true, username: cleanUsername };
2237: }

```

validateCredentialPair (server.mjs, lines 2239-2246)

validateCredentialPair part of authentication, authorization, or credential checking. In this file, it appears around lines 2239-2246 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2245: return { cleanUsername, cleanPassword };

Important local values include cleanUsername at line 2240; cleanPassword at line 2241. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2239: function validateCredentialPair(username, password) {
2240:   const cleanUsername = String(username || "").trim();
2241:   const cleanPassword = String(password || "").trim();
2242:   if ((cleanUsername && !cleanPassword) || (!cleanUsername && cleanPassword)) {
2243:     throw new Error("Both username and password/token are required when credentials are provided.");
2244:   }
2245:   return { cleanUsername, cleanPassword };
2246: }

```

normalizeGitBranchName (server.mjs, lines 2248-2327)

normalizeGitBranchName part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2248-2327 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, includes, endsWith, startsWith, detectRemoteDefaultBranch, runPublishGit, match, originRemoteExists, setOriginRemote, checkoutPublishBranch, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 2261: return ""; line 2263: return branch;; line 2267: if (!remoteUrl) return ""; line 2271: return normalizeGitBranchName(match?.[1] || "");. There are 6 additional return statements in the function body.

Important local values include branch at line 2249; result at line 2269; match at line 2270; cleanBranch at line 2295; current at line 2296; checkBranch at line 2307. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2248: function normalizeGitBranchName(value = "") {
2249:   const branch = String(value || "").trim();
2250:   if (
2251:     !branch ||
2252:     branch.length > 180 ||
2253:     branch.includes(".") ||
2254:     branch.endsWith(".") ||
2255:     branch.includes("@") ||
2256:     /[\\:\s~^*?*\[\]\x00-\x1f]/.test(branch) ||
2257:     branch.startsWith("/") ||
2258:     branch.endsWith("/") ||
2259:     branch.includes("//")
2260:   ) {
2261:     return "";
2262:   }

```

```

2263:   return branch;
2264: }
2265:
2266: async function detectRemoteDefaultBranch(remoteUrl = "") {
2267:   if (!remoteUrl) return "";
2268:   try {
2269:     const result = await runPublishGit(["ls-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 });
2270:     const match = String(result.stdout || "").match(/^ref:\s+refs\/heads\/(.+?)\s+HEAD$/m);
2271:     return normalizeGitBranchName(match?.[1] || "");
2272:   } catch {
2273:     return "";
2274:   }
2275: }
2276:
2277: async function originRemoteExists() {
2278:   try {
2279:     await runPublishGit(["remote", "get-url", "origin"]);
2280:     return true;
2281:   } catch {
2282:     return false;
2283:   }
2284: }
2285:
2286: async function setOriginRemote(remoteUrl) {
2287:   if (await originRemoteExists()) {
2288:     await runPublishGit(["remote", "set-url", "origin", remoteUrl]);
2289:   } else {
2290:     await runPublishGit(["remote", "add", "origin", remoteUrl]);
2291:   }
2292: }
2293:
2294: async function checkoutPublishBranch(branch) {
2295:   const cleanBranch = normalizeGitBranchName(branch) || "main";
2296:   const current = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2297:   if (current === cleanBranch) return cleanBranch;
2298:   try {
2299:     await runPublishGit(["checkout", cleanBranch]);
2300:   } catch {
2301:     await runPublishGit(["checkout", "-B", cleanBranch]);
2302:   }
2303:   return cleanBranch;
2304: }
2305:
2306: async function verifyPublishWriteAccess(access = {}) {
2307:   const checkBranch = `omb-auth-check-${safeSegment(os.hostname(), "device")}-${Date.now()}`;
2308:   try {
2309:     await runPublishGit(["push", "--dry-run", "origin", `HEAD:refs/heads/${checkBranch}`], {
2310:       timeout: 60 * 1000
2311:     });
2312:     return {
2313:       method: "temporary-dry-run-ref",
2314:       ref: checkBranch
2315:     };
2316:   } catch (error) {
2317:     throw publishAccessError(
2318:       "GitHub authorization is required before this website can be changed.",
2319:       gitFailureText(error),
2320:       access
2321:     );
2322:   }
2323: }
2324:
2325: async function ensurePublishHeadForWriteCheck() {
2326:   try {
2327:     await runPublishGit(["rev-parse", "--verify", "HEAD"]);

```

detectRemoteDefaultBranch (server.mjs, lines 2266-2275)

detectRemoteDefaultBranch part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2266-2275 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runPublishGit, match, and normalizeGitBranchName. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2267: if (!remoteUrl) return "";; line 2271: return normalizeGitBranchName(match?.[1] || "");; line 2273: return "";

Important local values include result at line 2269; match at line 2270. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2266: async function detectRemoteDefaultBranch(remoteUrl = "") {

```



```

2267:   if (!remoteUrl) return "";
2268:   try {
2269:     const result = await runPublishGit(["ls-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 });
2270:     const match = String(result.stdout || "").match(/^ref:\s+refs\/heads\/(.+)\s+HEAD$/m);
2271:     return normalizeGitBranchName(match?.[1] || "");
2272:   } catch {
2273:     return "";
2274:   }
2275: }

```

originRemoteExists (server.mjs, lines 2277-2284)

originRemoteExists part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2277-2284 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runPublishGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2280: return true;; line 2282: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2277: async function originRemoteExists() {
2278:   try {
2279:     await runPublishGit(["remote", "get-url", "origin"]);
2280:     return true;
2281:   } catch {
2282:     return false;
2283:   }
2284: }

```

setOriginRemote (server.mjs, lines 2286-2292)

setOriginRemote part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2286-2292 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references originRemoteExists, and runPublishGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2286: async function setOriginRemote(remoteUrl) {
2287:   if (await originRemoteExists()) {
2288:     await runPublishGit(["remote", "set-url", "origin", remoteUrl]);
2289:   } else {
2290:     await runPublishGit(["remote", "add", "origin", remoteUrl]);
2291:   }
2292: }

```

checkoutPublishBranch (server.mjs, lines 2294-2304)

checkoutPublishBranch part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2294-2304 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeGitBranchName, runPublishGit, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2297: if (current === cleanBranch) return cleanBranch;; line 2303: return cleanBranch;.

Important local values include cleanBranch at line 2295; current at line 2296. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2294: async function checkoutPublishBranch(branch) {
2295:   const cleanBranch = normalizeGitBranchName(branch) || "main";
2296:   const current = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2297:   if (current === cleanBranch) return cleanBranch;
2298:   try {
2299:     await runPublishGit(["checkout", cleanBranch]);
2300:   } catch {
2301:     await runPublishGit(["checkout", "-B", cleanBranch]);
2302:   }
2303:   return cleanBranch;
2304: }

```

verifyPublishWriteAccess (server.mjs, lines 2306-2323)

verifyPublishWriteAccess part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2306-2323 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references safeSegment, hostname, now, runPublishGit, publishAccessError, and gitFailureText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2312: return {

Important local values include checkBranch at line 2307. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2306: async function verifyPublishWriteAccess(access = {}) {
2307:   const checkBranch = `omb-auth-check-${safeSegment(os.hostname(), "device")}-${Date.now()}`;
2308:   try {
2309:     await runPublishGit(["push", "--dry-run", "origin", `HEAD:refs/heads/${checkBranch}`], {
2310:       timeout: 60 * 1000
2311:     });
2312:     return {
2313:       method: "temporary-dry-run-ref",
2314:       ref: checkBranch
2315:     };
2316:   } catch (error) {
2317:     throw publishAccessError(
2318:       "GitHub authorization is required before this website can be changed.",
2319:       gitFailureText(error),
2320:       access
2321:     );
2322:   }
2323: }

```

ensurePublishHeadForWriteCheck (server.mjs, lines 2325-2342)

ensurePublishHeadForWriteCheck part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2325-2342 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runPublishGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2328: return { created: false };; line 2340: return { created: true };.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2325: async function ensurePublishHeadForWriteCheck() {
2326:   try {
2327:     await runPublishGit(["rev-parse", "--verify", "HEAD"]);
2328:     return { created: false };
2329:   } catch {
2330:     await runPublishGit([
2331:       "-c",
2332:       "user.name=OMB Portfolio Builder",
2333:       "-c",
2334:       "user.email=omb-builder@localhost",
2335:       "commit",
2336:       "--allow-empty",
2337:       "-m",
2338:       "Initialize publish authorization check"
2339:     ]);
2340:     return { created: true };

```

```
2341:   }  
2342: }
```

synchronizePublishBranchFromRemote (server.mjs, lines 2344-2373)

synchronizePublishBranchFromRemote part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2344-2373 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeGitBranchName, runPublishGit, trim, checkoutPublishBranch, publishAccessError, and gitFailureText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2351: return {; line 2361: return {.

Important local values include cleanBranch at line 2345; remoteBranch at line 2347. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2344: async function synchronizePublishBranchFromRemote(branch, access = {}) {  
2345:   const cleanBranch = normalizeGitBranchName(branch) || "main";  
2346:   try {  
2347:     const remoteBranch = await runPublishGit(["ls-remote", "--heads", "origin", cleanBranch], {  
2348:       timeout: 45 * 1000  
2349:     });  
2350:     if (!String(remoteBranch.stdout || "").trim()) {  
2351:       return {  
2352:         branch: cleanBranch,  
2353:         remoteBranchExists: false,  
2354:         synchronized: false  
2355:       };  
2356:     }  
2357:     await runPublishGit(["fetch", "origin", cleanBranch], { timeout: 2 * 60 * 1000 });  
2358:     await checkoutPublishBranch(cleanBranch);  
2359:     await runPublishGit(["reset", "--hard", `origin/${cleanBranch}`], { timeout: 60 * 1000 });  
2360:     return {  
2361:       branch: cleanBranch,  
2362:       remoteBranchExists: true,  
2363:       synchronized: true  
2364:     };  
2365:   } catch (error) {  
2366:     throw publishAccessError(  
2367:       "The selected website target could not be synchronized from GitHub.",  
2368:       gitFailureText(error),  
2369:       { ...access, branch: cleanBranch }  
2370:     );  
2371:   }  
2372: }  
2373: }
```

capturePublishTargetState (server.mjs, lines 2375-2403)

capturePublishTargetState part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2375-2403 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runPublishGit, trim, readFile, and resolveInsidePortfolioRoot. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2402: return snapshot;.

Important local values include snapshot at line 2376. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2375: async function capturePublishTargetState() {  
2376:   const snapshot = {  
2377:     branch: "",  
2378:     customDomainExists: false,  
2379:     customDomainText: "",  
2380:     remote: "",  
2381:     remoteExists: false  
2382:   };  
2383:   try {  
2384:     snapshot.remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
```

```

2385:     snapshot.remoteExists = true;
2386:   } catch {
2387:     snapshot.remote = "";
2388:     snapshot.remoteExists = false;
2389:   }
2390:   try {
2391:     snapshot.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2392:   } catch {
2393:     snapshot.branch = "";
2394:   }
2395:   try {
2396:     snapshot.customDomainText = await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8");
2397:     snapshot.customDomainExists = true;
2398:   } catch {
2399:     snapshot.customDomainText = "";
2400:     snapshot.customDomainExists = false;
2401:   }
2402:   return snapshot;
2403: }

```

restorePublishTargetState (server.mjs, lines 2405-2429)

restorePublishTargetState part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2405-2429 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setOriginRemote, originRemoteExists, runPublishGit, checkoutPublishBranch, writeFile, resolveInsidePortfolioRoot, and rm. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2405: async function restorePublishTargetState(snapshot = {}) {
2406:   try {
2407:     if (snapshot.remoteExists && snapshot.remote) {
2408:       await setOriginRemote(snapshot.remote);
2409:     } else if (await originRemoteExists()) {
2410:       await runPublishGit(["remote", "remove", "origin"]);
2411:     }
2412:   } catch {
2413:     // Best-effort rollback; preserve the original error for the caller.
2414:   }
2415:   try {
2416:     if (snapshot.branch) await checkoutPublishBranch(snapshot.branch);
2417:   } catch {
2418:     // Best-effort rollback; preserve the original error for the caller.
2419:   }
2420:   try {
2421:     if (snapshot.customDomainExists) {
2422:       await writeFile(resolveInsidePortfolioRoot("CNAME"), snapshot.customDomainText, "utf8");
2423:     } else {
2424:       await rm(resolveInsidePortfolioRoot("CNAME"), { force: true });
2425:     }
2426:   } catch {
2427:     // Best-effort rollback; preserve the original error for the caller.
2428:   }
2429: }

```

writeTargetCustomDomain (server.mjs, lines 2431-2439)

writeTargetCustomDomain persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2431-2439 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resolveInsidePortfolioRoot, samePath, push, resolveInsideRoot, writeFile, and rm. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include targetPaths at line 2432. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2431: async function writeTargetCustomDomain(domain, customDomainProvided) {

```

```

2432:   const targetPaths = [resolveInsidePortfolioRoot("CNAME")];
2433:   if (!samePath(root, portfolioRoot)) targetPaths.push(resolveInsideRoot("CNAME"));
2434:   if (customDomainProvided && domain) {
2435:     for (const targetPath of targetPaths) await writeFile(targetPath, `${domain}\n`, "utf8");
2436:   } else if (customDomainProvided && !domain) {
2437:     for (const targetPath of targetPaths) await rm(targetPath, { force: true });
2438:   }
2439: }

```

ensureGitRepository (server.mjs, lines 2441-2454)

ensureGitRepository part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2441-2454 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mkdir, runPublishGit, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2445: if (insideWorkTree === "true") return;.

Important local values include insideWorkTree at line 2444; branch at line 2450. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2441: async function ensureGitRepository() {
2442:   await mkdir(portfolioRoot, { recursive: true });
2443:   try {
2444:     const insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
2445:     if (insideWorkTree === "true") return;
2446:   } catch {
2447:     await runPublishGit(["init"]);
2448:   }
2449:
2450:   const branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2451:   if (!branch) {
2452:     await runPublishGit(["checkout", "-B", "main"]);
2453:   }
2454: }

```

configurePublishTarget (server.mjs, lines 2456-2486)

This function accepts a publishing target, validates it, configures the local publish mirror repository, verifies write access, and saves the target only after authentication succeeds.

Publishing is dangerous if the target is wrong. This function protects the portfolio by making target setup explicit and verified before Apply to site can push changes.

It calls or references call, validatePublishRemoteUrl, validateCustomDomain, validateCredentialPair, ensureGitRepository, setOriginRemote, detectRemoteDefaultBranch, checkoutPublishBranch, writeTargetCustomDomain, getPublishTargetInfo, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2481: return {.

Important local values include customDomainProvided at line 2463; remoteUrl at line 2464; domain at line 2465; defaultBranch at line 2472; currentRemote at line 2478; credentials at line 2479; target at line 2480. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2456: async function configurePublishTarget(options = {}) {
2457:   const {
2458:     repositoryUrl = "",
2459:     customDomain = "",
2460:     authUsername = "",
2461:     authPassword = ""
2462:   } = options;
2463:   const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
2464:   const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
2465:   const domain = validateCustomDomain(customDomain);
2466:   validateCredentialPair(authUsername, authPassword);
2467:
2468:   await ensureGitRepository();
2469:
2470:   if (remoteUrl) {
2471:     await setOriginRemote(remoteUrl);
2472:     const defaultBranch = await detectRemoteDefaultBranch(remoteUrl);
2473:     if (defaultBranch) await checkoutPublishBranch(defaultBranch);
2474:   }

```

```

2475:
2476:   await writeTargetCustomDomain(domain, customDomainProvided);
2477:
2478:   const currentRemote = remoteUrl || (await getPublishTargetInfo()).remote;
2479:   const credentials = await storeGitCredentials(currentRemote, authUsername, authPassword);
2480:   const target = await getPublishTargetInfo();
2481:   return {
2482:     ...target,
2483:     credentialsStored: credentials.stored,
2484:     credentialUsername: credentials.username || ""
2485:   };
2486: }

```

readPublishAuthCache (server.mjs, lines 2488-2494)

readPublishAuthCache loads data from disk, network, browser storage, or a service. In this file, it appears around lines 2488-2494 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, and readFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2490: return JSON.parse(await readFile(publishAuthCachePath, "utf8"));; line 2492: return null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2488: async function readPublishAuthCache() {
2489:   try {
2490:     return JSON.parse(await readFile(publishAuthCachePath, "utf8"));
2491:   } catch {
2492:     return null;
2493:   }
2494: }

```

writePublishAuthCache (server.mjs, lines 2496-2535)

writePublishAuthCache persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2496-2535 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references readPublishAuthCache, getTime, isArray, map, parsePublishAuthTimestamp, filter, isFinite, toISOString, publishAuthCacheScope, writeFile, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2534: return cache;.

Important local values include previousCache at line 2497; checkedAt at line 2498; checkedAtMs at line 2499; sameTarget at line 2500; previousHistory at line 2504; successTimestamps at line 2507; successHistory at line 2514; successCountLastWeek at line 2515; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2496: async function writePublishAuthCache(access = {}) {
2497:   const previousCache = await readPublishAuthCache();
2498:   const checkedAt = new Date();
2499:   const checkedAtMs = checkedAt.getTime();
2500:   const sameTarget = previousCache
2501:     && previousCache.remote === access.remote
2502:     && previousCache.branch === access.branch
2503:     && previousCache.repository === access.repository;
2504:   const previousHistory = sameTarget && Array.isArray(previousCache.successHistory)
2505:     ? previousCache.successHistory
2506:     : [];
2507:   const successTimestamps = [
2508:     ...previousHistory
2509:       .map((entry) => parsePublishAuthTimestamp(entry))
2510:       .filter((timestamp) => Number.isFinite(timestamp))
2511:       .filter((timestamp) => checkedAtMs - timestamp < publishAuthExtendedTtlMs),
2512:     checkedAtMs
2513:   ];
2514:   const successHistory = successTimestamps.map((timestamp) => new Date(timestamp).toISOString());
2515:   const successCountLastWeek = successTimestamps
2516:     .filter((timestamp) => checkedAtMs - timestamp < publishAuthHistoryWindowMs)
2517:     .length;
2518:   const extendedTrust = successCountLastWeek > publishAuthExtendedThreshold;

```

```

2519:   const ttlMs = extendedTrust ? publishAuthExtendedTtlMs : publishAuthCacheTtlMs;
2520:   const cache = {
2521:     branch: access.branch || "",
2522:     checkedAt: checkedAt.toISOString(),
2523:     expiresAt: new Date(checkedAtMs + ttlMs).toISOString(),
2524:     extendedTrust,
2525:     remote: access.remote || "",
2526:     repository: access.repository || "",
2527:     scope: publishAuthCacheScope(),
2528:     successCountLastWeek,
2529:     successHistory,
2530:     trustDays: extendedTrust ? 30 : 1
2531:   };
2532:   await writeFile(publishAuthCachePath, `${JSON.stringify(cache, null, 2)}\n`, "utf8");
2533:   await chmod(publishAuthCachePath, 0o600).catch(() => {});
2534:   return cache;
2535: }

```

publishAuthCacheScope (server.mjs, lines 2537-2552)

publishAuthCacheScope part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2537-2552 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references userInfo, createHash, update, hostname, join, and digest. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2544: return createHash("sha256").

Important local values include username at line 2538. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2537: function publishAuthCacheScope() {
2538:   let username = process.env.USERNAME || process.env.USER || "";
2539:   try {
2540:     username = os.userInfo().username || username;
2541:   } catch {
2542:     // Environment username is a sufficient fallback for cache scoping.
2543:   }
2544:   return createHash("sha256")
2545:     .update([
2546:       os.hostname(),
2547:       username,
2548:       root,
2549:       portfolioRoot
2550:     ].join("\n"))
2551:     .digest("hex");
2552: }

```

parsePublishAuthTimestamp (server.mjs, lines 2554-2559)

parsePublishAuthTimestamp part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2554-2559 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, isFinite, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2556: if (Number.isFinite(direct)) return direct;; line 2558: return Date.parse(normalized);.

Important local values include direct at line 2555; normalized at line 2557. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2554: function parsePublishAuthTimestamp(value = "") {
2555:   const direct = Date.parse(String(value || ""));
2556:   if (Number.isFinite(direct)) return direct;
2557:   const normalized = String(value || "").replace(/(\.\d{3})\d+(Z|[-+]\d{2}:?\d{2})$/i, "$1$2");
2558:   return Date.parse(normalized);
2559: }

```

publishAuthCacheExpiresAt (server.mjs, lines 2561-2567)

publishAuthCacheExpiresAt part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2561-2567 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parsePublishAuthTimestamp, isFinite, and toISOString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2563: if (Number.isFinite(explicitExpiresAt)) return new Date(explicitExpiresAt).toISOString();; line 2565: if (!Number.isFinite(checkedException)) return "";; line 2566: return new Date(checkedException + publishAuthCacheTtlMs).toISOString();.

Important local values include explicitExpiresAt at line 2562; checkedAt at line 2564. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2561: function publishAuthCacheExpiresAt(cache) {
2562:   const explicitExpiresAt = parsePublishAuthTimestamp(cache?.expiresAt || "");
2563:   if (Number.isFinite(explicitExpiresAt)) return new Date(explicitExpiresAt).toISOString();
2564:   const checkedAt = parsePublishAuthTimestamp(cache?.checkedAt || "");
2565:   if (!Number.isFinite(checkedAt)) return "";
2566:   return new Date(checkedAt + publishAuthCacheTtlMs).toISOString();
2567: }
```

publishAuthCachelsFresh (server.mjs, lines 2569-2576)

This function decides whether a saved publishing authorization cache still applies to the current target. It checks scope, repository, branch, timestamp, and trust rules before reusing the cached authorization.

The user should not have to authenticate on every click, but the app also cannot trust stale credentials forever. This function is the balance between convenience and publishing safety.

It calls or references publishAuthCacheScope, parse, publishAuthCacheExpiresAt, isFinite, and now. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 2570: if (!cache || !access) return false;; line 2571: if (cache.remote !== access.remote || cache.branch !== access.branch || cache.repository !== access.repository) return false;; line 2572: if (cache.scope !== publishAuthCacheScope()) return false;; line 2574: if (!Number.isFinite(expiresAt)) return false;. There are 1 additional return statements in the function body.

Important local values include expiresAt at line 2573. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2569: function publishAuthCacheIsFresh(cache, access) {
2570:   if (!cache || !access) return false;
2571:   if (cache.remote !== access.remote || cache.branch !== access.branch || cache.repository !==
access.repository) return false;
2572:   if (cache.scope !== publishAuthCacheScope()) return false;
2573:   const expiresAt = Date.parse(publishAuthCacheExpiresAt(cache));
2574:   if (!Number.isFinite(expiresAt)) return false;
2575:   return Date.now() < expiresAt;
2576: }
```

assertPublishAccess (server.mjs, lines 2578-2669)

This function is the gatekeeper before website-changing operations. It checks whether target authentication exists and is still fresh enough, and it returns a clear access error when publishing is not allowed.

Apply to site should only run when the current user has proven write access. This function prevents accidental pushes from an unauthenticated or mismatched target state.

It calls or references runPublishGit, trim, publishAccessError, gitFailureText, remoteUrlForDisplay, workspaceHasCompatibleSiteFiles, parseGitHubRemote, readPublishAuthCache, publishAuthCacheIsFresh, publishAuthCacheExpiresAt, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2639: return {}; line 2656: return {}.

Important local values include insideWorkTree at line 2579; remote at line 2598; branch at line 2599; parsedRemote at line 2627; repository at line 2628; access at line 2630; cached at line 2637; remoteSync at line 2651; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2578: async function assertPublishAccess(options = {}) {
2579:   let insideWorkTree = "";
2580:   try {
2581:     insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
2582:   } catch (error) {
2583:     throw publishAccessError(
2584:       "This workspace is not connected to a Git repository.",
2585:       gitFailureText(error),
2586:       { repository: defaultSiteRepository }
2587:     );
2588:   }
2589:
2590:   if (insideWorkTree !== "true") {
2591:     throw publishAccessError(
2592:       "This workspace is local-only and cannot publish yet.",
2593:       "Clone or associate a GitHub Pages/static-site repository before applying to site.",
2594:       { repository: defaultSiteRepository }
2595:     );
2596:   }
2597:
2598:   let remote = "";
2599:   let branch = "";
2600:   try {
2601:     remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
2602:     branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2603:   } catch (error) {
2604:     throw publishAccessError(
2605:       "A publish remote or branch is missing.",
2606:       gitFailureText(error),
2607:       { repository: defaultSiteRepository }
2608:     );
2609:   }
2610:
2611:   if (!remote || !branch) {
2612:     throw publishAccessError(
2613:       "A publish remote or branch is missing.",
2614:       "Set the origin remote and use a named branch before applying to site.",
2615:       { remote: remoteUrlForDisplay(remote), branch }
2616:     );
2617:   }
2618:
2619:   if (options.requireCompatible !== false && !await workspaceHasCompatibleSiteFiles()) {
2620:     throw publishAccessError(
2621:       "This repository does not look like a compatible static portfolio website.",
2622:       "The workspace must include index.html, styles.css, and script.js before it can be used as a
publish target.",
2623:       { remote: remoteUrlForDisplay(remote), branch }
2624:     );
2625:   }
2626:
2627:   const parsedRemote = parseGitHubRemote(remote);
2628:   const repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` :
remoteUrlForDisplay(remote);
2629:
2630:   const access = {
2631:     branch,
2632:     remote: remoteUrlForDisplay(remote),
2633:     repository,
2634:     authorizationChecked: false
2635:   };
2636:
2637:   const cached = await readPublishAuthCache();
2638:   if (!options.force && publishAuthCacheIsFresh(cached, access)) {
2639:     return {
2640:       ...access,
2641:       authorizationCached: true,
2642:       authorizationChecked: true,
2643:       checkedAt: cached.checkedAt,
2644:       expiresAt: publishAuthCacheExpiresAt(cached),
2645:       extendedTrust: Boolean(cached.extendedTrust),
2646:       successCountLastWeek: cached.successCountLastWeek || 0,
2647:       trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1)
2648:     };
2649:   }
2650:
2651:   const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
2652:   const localHead = await ensurePublishHeadForWriteCheck();
2653:   const writeCheck = await verifyPublishWriteAccess(access);
2654:
2655:   const authCache = await writePublishAuthCache(access);
2656:   return {
2657:     ...access,
2658:     remoteSync,
2659:     localHead,
2660:     writeCheck,
2661:     authorizationChecked: true,
2662:     authorizationCached: false,
2663:     checkedAt: authCache.checkedAt,
2664:     expiresAt: authCache.expiresAt,
2665:     extendedTrust: Boolean(authCache.extendedTrust),
```

```

2666:     successCountLastWeek: authCache.successCountLastWeek || 0,
2667:     trustDays: authCache.trustDays || 1
2668:   };
2669: }

```

syncFromPublishTarget (server.mjs, lines 2671-2745)

This function loads compatible website files from the configured GitHub target into the local builder workspace. It pulls or fetches the target state, reads supported portfolio files, and updates local fields when the target is valid.

This is what lets the same owner move to another machine, authenticate, and pull the current website into the builder instead of starting blank.

It calls or references `assertPublishAccess`, `runPublishGit`, `trim`, `mkdtemp`, `join`, `tmpdir`, `runGit`, `fsAccess`, `push`, `includes`, and 11 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2735: `return {`.

Important local values include `publishAccess` at line 2672; `branch` at line 2673; `remote` at line 2674; `importRoot` at line 2675; `cloneRoot` at line 2676; `importPaths` at line 2677; `availablePaths` at line 2694; `backupRoot` at line 2712; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2671: async function syncFromPublishTarget() {
2672:   const publishAccess = await assertPublishAccess({ requireCompatible: false });
2673:   const branch = publishAccess.branch || "main";
2674:   const remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
2675:   const importRoot = await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-"));
2676:   const cloneRoot = path.join(importRoot, "target");
2677:   const importPaths = [
2678:     "projects.json",
2679:     "assets",
2680:     "docs",
2681:     "Backgrounds",
2682:     "CNAME",
2683:     ".nojekyll",
2684:     "robots.txt",
2685:     "sitemap.xml"
2686:   ];
2687:
2688:   try {
2689:     await runGit(["clone", "--depth", "1", "--branch", branch, remote, cloneRoot], {
2690:       cwd: importRoot,
2691:       timeout: 3 * 60 * 1000
2692:     });
2693:
2694:     const availablePaths = [];
2695:     for (const importPath of importPaths) {
2696:       try {
2697:         await fsAccess(path.join(cloneRoot, importPath));
2698:         availablePaths.push(importPath);
2699:       } catch {
2700:         // Optional path is absent in the selected site repository.
2701:       }
2702:     }
2703:
2704:     if (!availablePaths.includes("projects.json")) {
2705:       throw publishAccessError(
2706:         "The selected repository does not contain a projects.json portfolio catalog.",
2707:         "The target repository must contain a compatible OMB Portfolio Builder catalog before it can be
imported.",
2708:         publishAccess
2709:       );
2710:     }
2711:
2712:     const backupRoot = resolveInsideRoot(
2713:       ".omb-backups",
2714:       `load-target-${new Date().toISOString().replace(/[:.]/g, "-")}`
2715:     );
2716:     await mkdir(backupRoot, { recursive: true });
2717:
2718:     for (const importPath of availablePaths) {
2719:       const sourcePath = path.join(cloneRoot, importPath);
2720:       const targetPath = resolveInsideRoot(importPath);
2721:       try {
2722:         await fsAccess(targetPath);
2723:         await cp(targetPath, path.join(backupRoot, importPath), { recursive: true, force: true });
2724:         await rm(targetPath, { recursive: true, force: true });
2725:       } catch {
2726:         // Nothing local to back up for this path.
2727:       }
2728:       await cp(sourcePath, targetPath, { recursive: true, force: true });
2729:     }
2730:
2731:     const importedCatalog = await readFile(path.join(cloneRoot, "projects.json"), "utf8");

```

```

2732:     await writeFile(draftPath, importedCatalog, "utf8");
2733:     const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: false });
2734:
2735:     return {
2736:       ...publishAccess,
2737:       imported: availablePaths,
2738:       portfolioSync,
2739:       backup: path.relative(root, backupRoot),
2740:       draftUpdated: true
2741:     };
2742:   } finally {
2743:     await rm(importRoot, { recursive: true, force: true });
2744:   }
2745: }

```

authenticateGitHubForTarget (server.mjs, lines 2747-2843)

authenticateGitHubForTarget part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2747-2843 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call, validatePublishRemoteUrl, validateCustomDomain, validateCredentialPair, publishAccessError, getPublishTargetInfo, ensureGitRepository, capturePublishTargetState, setOriginRemote, getGitStatus, and 10 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2794: return {}; line 2831: return {}.

Important local values include customDomainProvided at line 2754; remoteUrl at line 2755; domain at line 2756; credentials at line 2757; snapshot at line 2767; status at line 2771; preliminaryTarget at line 2772; targetBranch at line 2781; plus 7 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2747: async function authenticateGitHubForTarget(options = {}) {
2748:   const {
2749:     repositoryUrl = "",
2750:     customDomain = "",
2751:     authUsername = "",
2752:     authPassword = ""
2753:   } = options;
2754:   const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
2755:   const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
2756:   const domain = validateCustomDomain(customDomain);
2757:   const credentials = validateCredentialPair(authUsername, authPassword);
2758:   if (!remoteUrl) {
2759:     throw publishAccessError(
2760:       "A GitHub repository URL is required before authentication.",
2761:       "Enter the GitHub Pages or compatible static-site repository URL, then click Authenticate target.",
2762:       await getPublishTargetInfo()
2763:     );
2764:   }
2765:
2766:   await ensureGitRepository();
2767:   const snapshot = await capturePublishTargetState();
2768:
2769:   try {
2770:     await setOriginRemote(remoteUrl);
2771:     const status = await getGitStatus();
2772:     const preliminaryTarget = await getPublishTargetInfo();
2773:     if (!status.git.ok) {
2774:       throw publishAccessError(
2775:         "Git for Windows is required for publishing.",
2776:         "Install Git for Windows, then reopen the builder and authenticate again.",
2777:         { ...preliminaryTarget, system: status }
2778:       );
2779:     }
2780:
2781:     const targetBranch = await detectRemoteDefaultBranch(remoteUrl) || "main";
2782:     await checkoutPublishBranch(targetBranch);
2783:
2784:     const targetBeforeAuth = await getPublishTargetInfo();
2785:     const cached = await readPublishAuthCache();
2786:     const cachedAccess = {
2787:       branch: targetBeforeAuth.branch,
2788:       remote: targetBeforeAuth.remote,
2789:       repository: targetBeforeAuth.repository
2790:     };
2791:     if (!credentials.cleanUsername && publishAuthCacheIsFresh(cached, cachedAccess)) {
2792:       await writeTargetCustomDomain(domain, customDomainProvided);
2793:       const target = await getPublishTargetInfo();
2794:       return {
2795:         ...cachedAccess,
2796:         branch: target.branch || cachedAccess.branch,

```

```

2797:         authorizationCached: true,
2798:         authorizationChecked: true,
2799:         checkedAt: cached.checkedAt,
2800:         expiresAt: publishAuthCacheExpiresAt(cached),
2801:         extendedTrust: Boolean(cached.extendedTrust),
2802:         successCountLastWeek: cached.successCountLastWeek || 0,
2803:         trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1),
2804:         credentialsStored: false,
2805:         credentialUsername: "",
2806:         system: status,
2807:         target
2808:     };
2809: }
2810:
2811: if (!status.credentialManager.ok && !credentials.cleanUsername) {
2812:     throw publishAccessError(
2813:         "Git Credential Manager is required for guided GitHub sign-in.",
2814:         "Install or repair Git for Windows with Git Credential Manager enabled, or provide a GitHub
username and personal access token.",
2815:         { ...targetBeforeAuth, system: status }
2816:     );
2817: }
2818:
2819: const storedCredentials = await storeGitCredentials(remoteUrl, authUsername, authPassword);
2820: if (!storedCredentials.stored) {
2821:     await runGit(["credential-manager", "github", "login"], {
2822:         cwd: portfolioRoot,
2823:         timeout: 5 * 60 * 1000,
2824:         windowsHide: false
2825:     });
2826: }
2827:
2828: const access = await assertPublishAccess({ force: true, requireCompatible: false });
2829: await writeTargetCustomDomain(domain, customDomainProvided);
2830: const target = await getPublishTargetInfo();
2831: return {
2832:     ...access,
2833:     branch: target.branch || access.branch,
2834:     credentialsStored: storedCredentials.stored,
2835:     credentialUsername: storedCredentials.username || "",
2836:     system: await getGitStatus(),
2837:     target
2838: };
2839: } catch (error) {
2840:     await restorePublishTargetState(snapshot);
2841:     throw error;
2842: }
2843: }

```

installGitForWindows (server.mjs, lines 2845-2868)

installGitForWindows part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2845-2868 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runOptionalCommand, spawn, and unref. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2849: return {}; line 2863: return {}.

Important local values include winget at line 2846; downloadUrl at line 2847; child at line 2856. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2845: async function installGitForWindows() {
2846:     const winget = await runOptionalCommand("winget", ["--version"]);
2847:     const downloadUrl = "https://git-scm.com/download/win";
2848:     if (!winget.ok) {
2849:         return {
2850:             launched: false,
2851:             downloadUrl,
2852:             message: "winget was not found. Open the Git for Windows download page and install Git with Git
Credential Manager enabled."
2853:         };
2854:     }
2855:
2856:     const child = spawn("winget", ["install", "--id", "Git.Git", "-e", "--source", "winget"], {
2857:         detached: true,
2858:         shell: true,
2859:         stdio: "ignore",
2860:         windowsHide: false
2861:     });
2862:     child.unref();
2863:     return {

```

```

2864:     launched: true,
2865:     downloadUrl,
2866:     message: "The Git for Windows installer was launched through winget. Follow the installer prompts,
then reopen the builder."
2867:   };
2868: }

```

getUpdateInfo (server.mjs, lines 2870-2906)

getUpdateInfo part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2870-2906 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, Error, json, replace, find, has, and compareVersions. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2884: return {}; line 2899: return {}.

Important local values include currentVersion at line 2871; owner at line 2872; repo at line 2873; response at line 2875; release at line 2879; latestVersion at line 2880; installer at line 2881; portable at line 2882; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2870: async function getUpdateInfo() {
2871:   const currentVersion = process.env.OMB_APP_VERSION || packageJson.version || "0.0.0";
2872:   const owner = "otienomaurice";
2873:   const repo = "omb-portfolio-builder";
2874:   try {
2875:     const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
2876:       headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
2877:     });
2878:     if (!response.ok) throw new Error(`GitHub returned ${response.status}`);
2879:     const release = await response.json();
2880:     const latestVersion = String(release.tag_name || "").replace(/^builder-v/i, "");
2881:     const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
2882:     const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
2883:     const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
2884:     return {
2885:       ok: true,
2886:       currentVersion,
2887:       latestVersion,
2888:       updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
2889:       updateBlocked,
2890:       blockedReason: updateBlocked
2891:         ? `Builder ${latestVersion} is skipped because its installer can hang while running the old
uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
2892:         : "",
2893:       releaseUrl: release.html_url || "",
2894:       installerUrl: installer?.browser_download_url || "",
2895:       installerName: installer?.name || "",
2896:       portableUrl: portable?.browser_download_url || ""
2897:     };
2898:   } catch (error) {
2899:     return {
2900:       ok: false,
2901:       currentVersion,
2902:       latestVersion: currentVersion,
2903:       updateAvailable: false,
2904:       error: error.message || "Could not check for updates."
2905:     };
2906:   }

```

installer (server.mjs, lines 2881-2897)

installer part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2881-2897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, has, and compareVersions. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2884: return {}.

Important local values include installer at line 2881; portable at line 2882; updateBlocked at line 2883. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2881:     const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
2882:     const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
2883:     const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
2884:     return {
2885:       ok: true,
2886:       currentVersion,
2887:       latestVersion,
2888:       updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
2889:       updateBlocked,
2890:       blockedReason: updateBlocked
2891:       ? `Builder ${latestVersion} is skipped because its installer can hang while running the old
uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
2892:       : "",
2893:       releaseUrl: release.html_url || "",
2894:       installerUrl: installer?.browser_download_url || "",
2895:       installerName: installer?.name || "",
2896:       portableUrl: portable?.browser_download_url || ""
2897:     };
```

portable (server.mjs, lines 2882-2897)

portable part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2882-2897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, has, and compareVersions. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2884: return {.

Important local values include portable at line 2882; updateBlocked at line 2883. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2882:     const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
2883:     const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
2884:     return {
2885:       ok: true,
2886:       currentVersion,
2887:       latestVersion,
2888:       updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
2889:       updateBlocked,
2890:       blockedReason: updateBlocked
2891:       ? `Builder ${latestVersion} is skipped because its installer can hang while running the old
uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
2892:       : "",
2893:       releaseUrl: release.html_url || "",
2894:       installerUrl: installer?.browser_download_url || "",
2895:       installerName: installer?.name || "",
2896:       portableUrl: portable?.browser_download_url || ""
2897:     };
```

getBuilderReleaseDownloadReport (server.mjs, lines 2909-2914)

getBuilderReleaseDownloadReport part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2909-2914 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include owner at line 2910; repo at line 2911; response at line 2912. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2909: async function getBuilderReleaseDownloadReport() {
2910:   const owner = "otienomaurice";
2911:   const repo = "omb-portfolio-builder";
2912:   const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
2913:     headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
2914:   });
```

```
2914: });
```

assets (server.mjs, lines 2917-2923)

assets part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2917-2923 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include assets at line 2917. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2917: const assets = (release.assets || []).map((asset) => ({
2918:   name: asset.name || "",
2919:   downloadCount: Number(asset.download_count || 0),
2920:   sizeBytes: Number(asset.size || 0),
2921:   updatedAt: asset.updated_at || "",
2922:   url: asset.browser_download_url || ""
2923: }));
```

getSecurityReport (server.mjs, lines 2933-2977)

getSecurityReport part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2933-2977 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references allSettled, getBuilderReleaseDownloadReport, readPublishAuthCache, getPublishTargetInfo, toISOString, publishAuthCachelsFresh, and publishAuthCacheExpiresAt. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2941: return {.

Important local values include authCache at line 2939; target at line 2940. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2933: async function getSecurityReport() {
2934:   const [downloadResult, authCacheResult, targetResult] = await Promise.allSettled([
2935:     getBuilderReleaseDownloadReport(),
2936:     readPublishAuthCache(),
2937:     getPublishTargetInfo()
2938:   ]);
2939:   const authCache = authCacheResult.status === "fulfilled" ? authCacheResult.value : null;
2940:   const target = targetResult.status === "fulfilled" ? targetResult.value : null;
2941:   return {
2942:     ok: true,
2943:     generatedAt: new Date().toISOString(),
2944:     appDownloads: downloadResult.status === "fulfilled"
2945:       ? { ok: true, ...downloadResult.value }
2946:       : { ok: false, error: downloadResult.reason?.message || "GitHub release download counts were not
available." },
2947:     publishingAuth: {
2948:       targetRepository: target?.repository || "",
2949:       branch: target?.branch || "",
2950:       authenticated: Boolean(authCache && publishAuthCacheIsFresh(authCache, {
2951:         remote: target?.remote || "",
2952:         repository: target?.repository || "",
2953:         branch: target?.branch || ""
2954:       })),
2955:       expiresAt: publishAuthCacheExpiresAt(authCache) || "",
2956:       trustDays: Number(authCache?.trustDays || 0),
2957:       successCountLastWeek: Number(authCache?.successCountLastWeek || 0)
2958:     },
2959:     websiteAccess: {
2960:       available: false,
2961:       summary: "Visitor counts and visitor IP addresses are not available from a static GitHub Pages site
or GitHub release download counts.",
2962:       recommendedSetup: [
2963:         "Proxy the custom domain through Cloudflare instead of DNS-only.",
2964:         "Enable Cloudflare Web Analytics for aggregate page-view counts.",
```

```

2965:         "Use Cloudflare Logpush, Analytics Engine, or a Worker-backed endpoint if you need IP-level
security logs.",
2966:         "Publish a privacy notice before collecting raw IP addresses or persistent identifiers."
2967:     ],
2968: },
2969: securityControls: [
2970:     "Local builder write APIs are restricted to local requests.",
2971:     "Publishing target details are only returned to the local machine.",
2972:     "GitHub publishing authorization is cached for one day by default, or longer after repeated
successful authorizations.",
2973:     "The desktop app runs with Electron nodeIntegration disabled, contextIsolation enabled, and sandbox
enabled.",
2974:     "Deployable website security headers and security.txt metadata are included in the publishable site
files."
2975: ],
2976: };
2977: }

```

safeUpdateFileSegment (server.mjs, lines 2979-2984)

safeUpdateFileSegment validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2979-2984 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2980: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2979: function safeUpdateFileSegment(value = "") {
2980:     return String(value || "")
2981:         .replace(/^[a-z0-9._-]+/gi, "-")
2982:         .replace(/^-+|-+$/g, "")
2983:         .slice(0, 80) || "latest";
2984: }

```

powershellSingleQuoted (server.mjs, lines 2986-2988)

powershellSingleQuoted part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2986-2988 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2987: return `\${String(value).replaceAll("'", "'')}";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2986: function powershellSingleQuoted(value = "") {
2987:     return `${String(value).replaceAll("'", "'')}";
2988: }

```

downloadAndLaunchAppUpdate (server.mjs, lines 2990-3165)

This function downloads a selected app installer release, writes an update handoff script, closes the running app, starts the installer, and attempts to relaunch the updated executable afterward.

Windows cannot cleanly overwrite a running app. This function coordinates update work so the user sees a simple Update button instead of manually closing, installing, and reopening everything.

It calls or references getUpdateInfo, Error, join, tmpdir, mkdir, safeUpdateFileSegment, fetch, from, arrayBuffer, writeFile, and 20 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 3056: " if (\$candidate -and (Test-Path -LiteralPath \$candidate)) { return \$candidate }";, line 3060: " return \$null";, line 3090: " if (-not \$process) { return 0 }";, line 3094: " return 124";. There are 2 additional

return statements in the function body.

Important local values include update at line 2991; updateRoot at line 3006; installerName at line 3008; installerPath at line 3011; response at line 3013; installerBuffer at line 3017; updateLogPath at line 3023; currentExecutable at line 3024; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2990: async function downloadAndLaunchAppUpdate() {
2991:   const update = await getUpdateInfo();
2992:   if (!update.ok) throw new Error(update.error || "Could not check for updates.");
2993:   if (update.updateBlocked) {
2994:     throw new Error(update.blockedReason || `Builder ${update.latestVersion} is temporarily blocked from
automatic updates.`);
2995:   }
2996:   if (!update.updateAvailable) {
2997:     throw new Error(`OMB Portfolio Builder ${update.currentVersion || ""} is already up to date.`);
2998:   }
2999:   if (!update.installerUrl) {
3000:     throw new Error("The latest GitHub Release does not include a Windows setup installer.");
3001:   }
3002:   if (process.platform !== "win32") {
3003:     throw new Error("Automatic installer updates are currently available only on Windows.");
3004:   }
3005:
3006:   const updateRoot = path.join(os.tmpdir(), "omb-portfolio-builder-updates");
3007:   await mkdir(updateRoot, { recursive: true });
3008:   const installerName = update.installerName && /\.exe$/i.test(update.installerName)
3009:     ? update.installerName
3010:     : `OMB-Portfolio-Builder-Setup-${safeUpdateFileSegment(update.latestVersion)}-x64.exe`;
3011:   const installerPath = path.join(updateRoot, installerName);
3012:
3013:   const response = await fetch(update.installerUrl, {
3014:     headers: { "Accept": "application/octet-stream", "User-Agent": "OMB-Portfolio-Builder" }
3015:   });
3016:   if (!response.ok) throw new Error(`GitHub returned ${response.status} while downloading the
installer.`);
3017:   const installerBuffer = Buffer.from(await response.arrayBuffer());
3018:   if (installerBuffer.length < 1024 * 1024) {
3019:     throw new Error("The downloaded installer was unexpectedly small, so the update was stopped.");
3020:   }
3021:   await writeFile(installerPath, installerBuffer);
3022:
3023:   const updateLogPath = path.join(updateRoot,
`update-${safeUpdateFileSegment(update.latestVersion)}.log`);
3024:   const currentExecutable = /OMB Portfolio Builder\.exe$/i.test(process.execPath || "") ?
process.execPath : "";
3025:   const currentInstallDirectory = currentExecutable ? path.dirname(currentExecutable) : "";
3026:   const localAppDataExecutable = process.env.LOCALAPPDATA
3027:     ? path.join(process.env.LOCALAPPDATA, "Programs", "OMB Portfolio Builder", "OMB Portfolio
Builder.exe")
3028:     : "";
3029:   const legacyManagedApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio
Builder", "OMB Portfolio Builder.exe");
3030:   const legacyFlatApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio
Builder.exe");
3031:   const relaunchCandidates = Array.from(new Set([
3032:     localAppDataExecutable,
3033:     currentExecutable,
3034:     currentInstallDirectory ? path.join(currentInstallDirectory, "OMB Portfolio Builder.exe") : "",
3035:     legacyFlatApplicationExecutable,
3036:     legacyManagedApplicationExecutable,
3037:     process.env.ProgramFiles ? path.join(process.env.ProgramFiles, "OMB Portfolio Builder", "OMB
Portfolio Builder.exe") : "",
3038:     process.env["ProgramFiles(x86)"] ? path.join(process.env["ProgramFiles(x86)"], "OMB Portfolio
Builder", "OMB Portfolio Builder.exe") : ""
3039:   ].filter(Boolean)));
3040:   const relaunchCandidatesPs = `@(${relaunchCandidates.map(powershellSingleQuoted).join(", ")});`;
3041:   const launcherPath = path.join(updateRoot, `run-
update-${safeUpdateFileSegment(update.latestVersion)}.ps1`);
3042:   const launcherCommandPath = path.join(updateRoot, `run-
update-${safeUpdateFileSegment(update.latestVersion)}.cmd`);
3043:   const launcherScript = [
3044:     "ErrorActionPreference = 'Continue'",
3045:     `$parentPid = ${process.pid}`,
3046:     `$installer = ${powershellSingleQuoted(installerPath)}`,
3047:     `$log = ${powershellSingleQuoted(updateLogPath)}`,
3048:     `$candidates = ${relaunchCandidatesPs}`,
3049:     "function Write-UpdateLog([string]$Message) {",
3050:     "  try { Add-Content -LiteralPath $log -Value ((Get-Date).ToString('s') + ' ' + $Message) } catch",
3051:     "}",
3052:     "function Wait-ForExecutable([string[]]$Paths, [int]$Seconds) {",
3053:     "  $deadline = (Get-Date).AddSeconds($Seconds)",
3054:     "  do {",
3055:     "    foreach ($candidate in $Paths) {",
3056:     "      if ($candidate -and (Test-Path -LiteralPath $candidate)) { return $candidate }",
3057:     "    }",
3058:     "    Start-Sleep -Seconds 1",
3059:     "  } while ((Get-Date) -lt $deadline)",
```

```

3060:     " return $null",
3061:     "}",
3062:     "function Stop-BuilderProcesses([int]$ProcessId, [string[]]$Paths) {"",
3063:     " try {"",
3064:     "     $normalizedPaths = @($Paths | Where-Object { $_ } | ForEach-Object {"",
3065:     "         try { [System.IO.Path]::GetFullPath($_).ToLowerInvariant() } catch { $_.ToLowerInvariant() }",
3066:     "     }",
3067:     "     $processes = Get-CimInstance Win32_Process | Where-Object {"",
3068:     "         $_.ProcessId -eq $ProcessId -or",
3069:     "         $_.Name -eq 'OMB Portfolio Builder.exe' -or",
3070:     "         ($_.ExecutablePath -and ($normalizedPaths -contains",
3071:     "             ([System.IO.Path]::GetFullPath($_.ExecutablePath).ToLowerInvariant()))",
3072:     "     } | Sort-Object ProcessId -Unique",
3073:     "     foreach ($process in $processes) {"",
3074:     "         try {"",
3075:     "             Write-UpdateLog ('Stopping previous builder process PID ' + $process.ProcessId + ' at ' +",
3076:     "                 $process.ExecutablePath)",
3077:     "             Stop-Process -Id $process.ProcessId -Force -ErrorAction SilentlyContinue",
3078:     "         } catch { Write-UpdateLog ('Could not stop PID ' + $process.ProcessId + ': ' +",
3079:     "             $_.Exception.Message) }",
3080:     "     }",
3081:     "     Start-Sleep -Seconds 2",
3082:     " } catch { Write-UpdateLog ('Could not stop previous builder processes: ' + $_.Exception.Message)",
3083:     " }",
3084:     "}",
3085:     "function Run-Installer([switch]$Elevated) {"",
3086:     "     $process = $null",
3087:     "     if ($Elevated) {"",
3088:     "         Write-UpdateLog 'Starting installer with Windows elevation prompt.'",
3089:     "         $process = Start-Process -FilePath $installer -ArgumentList @('/S') -Verb RunAs -PassThru",
3090:     "     } else {"",
3091:     "         Write-UpdateLog 'Starting installer silently.'",
3092:     "         $process = Start-Process -FilePath $installer -ArgumentList @('/S') -WindowStyle Hidden",
3093:     "     }",
3094:     "     if (-not $process) { return 0 }",
3095:     "     if (-not $process.WaitForExit(900000)) {"",
3096:     "         Write-UpdateLog ('Installer timed out after 15 minutes. PID: ' + $process.Id)",
3097:     "         try { Stop-Process -Id $process.Id -Force -ErrorAction SilentlyContinue } catch { Write-",
3098:     "             UpdateLog ('Could not stop timed-out installer: ' + $_.Exception.Message) }",
3099:     "         return 124",
3100:     "     }",
3101:     "     return $process.ExitCode",
3102:     " }",
3103:     "Write-UpdateLog 'Update launcher started.'",
3104:     "Start-Sleep -Seconds 3",
3105:     "try { Wait-Process -Id $parentPid -Timeout 8; Write-UpdateLog 'Previous app process exited.' } catch",
3106:     "{ Write-UpdateLog ('Previous app wait finished: ' + $_.Exception.Message) }",
3107:     "Stop-BuilderProcesses $parentPid $candidates",
3108:     "Start-Sleep -Seconds 1",
3109:     "$installerProcess = $null",
3110:     "$installerExit = 1",
3111:     "try {"",
3112:     "     Write-UpdateLog ('Starting installer: ' + $installer)",
3113:     "     $installerExit = Run-Installer",
3114:     "     Write-UpdateLog ('Installer exit code: ' + $installerExit)",
3115:     " } catch {"",
3116:     "     ... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

syncPortfolioPublishFiles (server.mjs, lines 3167-3200)

syncPortfolioPublishFiles part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 3167-3200 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references samePath, mkdir, resolveInsideRoot, resolveInsidePortfolioRoot, pathExists, rm, dirname, cp, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3169: return { copied: [], skipped: [], workspace: portfolioRoot, separatedWorkspace: false };; line 3194: return {.

Important local values include copied at line 3173; skipped at line 3174; sourcePath at line 3179; targetPath at line 3180. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3167: async function syncPortfolioPublishFiles(options = {}) {
3168:   if (samePath(root, portfolioRoot)) {
3169:     return { copied: [], skipped: [], workspace: portfolioRoot, separatedWorkspace: false };
3170:   }
3171:
3172:   const { removeMissing = false } = options;
3173:   const copied = [];
3174:   const skipped = [];

```

```

3175:
3176:   await mkdir(portfolioRoot, { recursive: true });
3177:
3178:   for (const relativePath of publishPaths) {
3179:     const sourcePath = resolveInsideRoot(relativePath);
3180:     const targetPath = resolveInsidePortfolioRoot(relativePath);
3181:     if (await pathExists(sourcePath)) {
3182:       await rm(targetPath, { recursive: true, force: true });
3183:       await mkdir(path.dirname(targetPath), { recursive: true });
3184:       await cp(sourcePath, targetPath, { recursive: true, force: true });
3185:       copied.push(relativePath);
3186:     } else if (removeMissing) {
3187:       await rm(targetPath, { recursive: true, force: true });
3188:       skipped.push(relativePath);
3189:     } else {
3190:       skipped.push(relativePath);
3191:     }
3192:   }
3193:
3194:   return {
3195:     copied,
3196:     skipped,
3197:     workspace: portfolioRoot,
3198:     separatedWorkspace: true
3199:   };
3200: }

```

publishSiteChanges (server.mjs, lines 3202-3238)

This function applies the generated portfolio output to the publish mirror, stages allowed files, commits the changes, and pushes them to the configured website repository.

This is the final local-to-public handoff. It keeps publishing constrained to generated website files so the builder app does not blindly push unrelated local files.

It calls or references `assertPublishAccess`, `runPublishGit`, `trim`, `synchronizePublishBranchFromRemote`, `syncPortfolioPublishFiles`, `bumpPublishedAssetVersions`, `stageablePublishPaths`, `Error`, `toISOString`, and `slice`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3227: `return {`.

Important local values include `access` at line 3203; `branch` at line 3204; `remoteSync` at line 3205; `portfolioSync` at line 3206; `stageablePaths` at line 3209; `status` at line 3215; `hasChanges` at line 3216; `commit` at line 3218; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3202: async function publishSiteChanges(publishAccess = null) {
3203:   const access = publishAccess || await assertPublishAccess();
3204:   const branch = access.branch || (await runPublishGit(["branch", "--show-current"])).stdout.trim();
3205:   const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
3206:   const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: true });
3207:   await bumpPublishedAssetVersions(portfolioRoot);
3208:
3209:   const stageablePaths = await stageablePublishPaths(publishPaths);
3210:   if (!stageablePaths.length) {
3211:     throw new Error("No compatible site files were available to publish.");
3212:   }
3213:
3214:   await runPublishGit(["add", "-A", "--", ...stageablePaths]);
3215:   const status = await runPublishGit(["status", "--porcelain", "--", ...stageablePaths]);
3216:   const hasChanges = status.stdout.trim().length > 0;
3217:
3218:   let commit = null;
3219:   if (hasChanges) {
3220:     const message = `Update portfolio site ${new Date().toISOString().slice(0, 10)}`;
3221:     commit = await runPublishGit(["commit", "-m", message]);
3222:   }
3223:
3224:   const pushArgs = branch ? ["push", "origin", branch] : ["push"];
3225:   const push = await runPublishGit(pushArgs);
3226:
3227:   return {
3228:     ...access,
3229:     portfolioWorkspace: portfolioRoot,
3230:     remoteSync,
3231:     portfolioSync,
3232:     branch: branch || "current branch",
3233:     committed: hasChanges,
3234:     commitOutput: commit?.stdout || commit?.stderr || "",
3235:     pushed: true,
3236:     pushOutput: push.stdout || push.stderr || ""
3237:   };
3238: }

```

handlePortfolioAi (server.mjs, lines 3240-3322)

This backend handler receives AI chat requests, classifies the question, builds the model payload, chooses an available AI route, and falls back to local rule-based answers when remote AI is unavailable.

It provides the conversational intelligence layer while keeping secrets and model calls away from static frontend code.

It calls or references readRequestJson, clampText, trim, cleanConversationHistory, Set, has, sendJson, enrichPortfolioContext, callOllamaPortfolioAi, toLowerCase, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3250: return;; line 3265: return;; line 3268: return;; line 3321: return { data, openAiResponse, selectedModel };;.

Important local values include body at line 3241; question at line 3242; conversation at line 3243; validIntents at line 3244; intent at line 3245; allowWebSearch at line 3246; context at line 3253; apiKey at line 3255; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3240: async function handlePortfolioAi(request, response) {
3241:   const body = await readRequestJson(request);
3242:   const question = clampText(body.question, 1200).trim();
3243:   const conversation = cleanConversationHistory(body.conversation);
3244:   const validIntents = new Set(["general_conversation", "general_engineering", "general_knowledge",
"portfolio_specific"]);
3245:   const intent = validIntents.has(body.intent) ? body.intent : "portfolio_specific";
3246:   const allowWebSearch = body.allowWebSearch === true;
3247:
3248:   if (!question) {
3249:     sendJson(response, 400, { error: "Question is required." });
3250:     return;
3251:   }
3252:
3253:   const context = await enrichPortfolioContext({ ...(body.context || {}), question });
3254:
3255:   const apiKey = process.env.OPENAI_API_KEY || "";
3256:   if (!apiKey) {
3257:     const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch
});
3258:     if (ollama.ok) {
3259:       sendJson(response, 200, {
3260:         answer: ollama.answer,
3261:         model: ollama.model,
3262:         provider: "ollama",
3263:         usedWebSearch: false
3264:       });
3265:       return;
3266:     }
3267:     sendJson(response, 503, { error: ollama.error || "No local Ollama model or OPENAI_API_KEY is
available for the local backend." });
3268:     return;
3269:   }
3270:
3271:   const model = process.env.OPENAI_MODEL || "gpt-5.4";
3272:   const fallbackModel = process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1";
3273:   const webSearchMode = String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase();
3274:   const enableWebSearch = webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch);
3275:   const buildPayload = (selectedModel) => ({
3276:     model: selectedModel,
3277:     input: [
3278:       {
3279:         role: "developer",
3280:         content: [{ type: "input_text", text: portfolioAiInstructions }]
3281:       },
3282:       {
3283:         role: "user",
3284:         content: [{
3285:           type: "input_text",
3286:           text: [
3287:             `Visitor question: ${question}`,
3288:             `Question intent: ${intent}`,
3289:             `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
3290:             "",
3291:             "Recent conversation JSON:",
3292:             clampText(JSON.stringify(conversation, null, 2), 8000),
3293:             "",
3294:             "Portfolio context JSON:",
3295:             clampText(JSON.stringify(context, null, 2), 18000)
3296:           ].join("\n")
3297:         }]
3298:       }
3299:     ],
3300:     max_output_tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
3301:     reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
3302:     text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
3303:   });
```

```

3304:
3305:   const callModel = async (selectedModel) => {
3306:     const payload = buildPayload(selectedModel);
3307:     if (enableWebSearch) {
3308:       payload.tools = [{ type: "web_search", search_context_size:
process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE || "low" }];
3309:     }
3310:
3311:     const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
3312:       method: "POST",
3313:       headers: {
3314:         "Authorization": `Bearer ${apiKey}`,
3315:         "Content-Type": "application/json"
3316:       },
3317:       body: JSON.stringify(payload)
3318:     });
3319:
3320:     const data = await openAiResponse.json().catch(() => ({}));
3321:     return { data, openAiResponse, selectedModel };
3322:   };

```

buildPayload (server.mjs, lines 3275-3303)

buildPayload creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 3275-3303 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clampText, stringify, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include buildPayload at line 3275. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3275:   const buildPayload = (selectedModel) => ({
3276:     model: selectedModel,
3277:     input: [
3278:       {
3279:         role: "developer",
3280:         content: [{ type: "input_text", text: portfolioAiInstructions }]
3281:       },
3282:       {
3283:         role: "user",
3284:         content: [{
3285:           type: "input_text",
3286:           text: [
3287:             `Visitor question: ${question}`,
3288:             `Question intent: ${intent}`,
3289:             `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
3290:             "",
3291:             "Recent conversation JSON:",
3292:             clampText(JSON.stringify(conversation, null, 2), 8000),
3293:             "",
3294:             "Portfolio context JSON:",
3295:             clampText(JSON.stringify(context, null, 2), 18000)
3296:           ].join("\n")
3297:         }
3298:       ]
3299:     },
3300:     max_output_tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
3301:     reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
3302:     text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
3303:   });

```

callModel (server.mjs, lines 3305-3318)

callModel part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 3305-3318 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references async, buildPayload, fetch, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include `callModel` at line 3305; `payload` at line 3306; `openAiResponse` at line 3311. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3305: const callModel = async (selectedModel) => {
3306:   const payload = buildPayload(selectedModel);
3307:   if (enableWebSearch) {
3308:     payload.tools = [{ type: "web_search", search_context_size:
process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE || "low" }];
3309:   }
3310:
3311:   const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
3312:     method: "POST",
3313:     headers: {
3314:       "Authorization": `Bearer ${apiKey}`,
3315:       "Content-Type": "application/json"
3316:     },
3317:     body: JSON.stringify(payload)
3318:   });
```

handleApi (server.mjs, lines 3342-3602)

This is the local backend router. It reads the request path, selects the matching API behavior, calls the responsible function, and sends the response back to the builder frontend.

Almost every builder action crosses this function. It is the traffic director between button clicks and backend work.

It calls or references `readJsonFile`, `sendJson`, `join`, `isLocalRequest`, `getPublishTargetInfo`, `getGitStatus`, `getUpdateInfo`, `getSecurityReport`, `compileToolStatus`, `handlePortfolioAi`, and 20 more. Endpoint references found inside it are `/api/app-update`, `/api/app-update/install`, `/api/apply-catalog`, `/api/catalog`, `/api/code/beautify`, `/api/code/compile`, `/api/code/install-tools`, `/api/code/save`, `/api/code/tools`, `/api/github-authenticate`, and 9 more. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 3350: `return true`; line 3355: `return true`; line 3361: `return true`; line 3364: `return true`. There are 8 additional return statements in the function body.

Important local values include `catalog` at line 3345; `body` at line 3409; `language` at line 3410; `body` at line 3424; `saved` at line 3425; `body` at line 3435; `result` at line 3436; `body` at line 3446; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3342: async function handleApi(request, response, url) {
3343:   if (request.method === "GET" && url.pathname === "/api/catalog") {
3344:     try {
3345:       const catalog = await readJsonFile(draftPath);
3346:       sendJson(response, 200, { source: "draft", catalog });
3347:     } catch {
3348:       sendJson(response, 200, { source: "published", catalog: await readJsonFile(catalogPath) });
3349:     }
3350:     return true;
3351:   }
3352:
3353:   if (request.method === "GET" && url.pathname === "/api/templates") {
3354:     sendJson(response, 200, await readJsonFile(path.join(root, "project-templates.json")));
3355:     return true;
3356:   }
3357:
3358:   if (request.method === "GET" && url.pathname === "/api/publish-target") {
3359:     if (!isLocalRequest(request)) {
3360:       sendJson(response, 403, { error: "Publishing target details are only available from this computer." });
3361:       return true;
3362:     }
3363:     sendJson(response, 200, { ok: true, target: await getPublishTargetInfo() });
3364:     return true;
3365:   }
3366:
3367:   if (request.method === "GET" && url.pathname === "/api/system-check") {
3368:     sendJson(response, 200, { ok: true, system: await getGitStatus(), target: await
getPublishTargetInfo() });
3369:     return true;
3370:   }
3371:
3372:   if (request.method === "GET" && url.pathname === "/api/app-update") {
3373:     sendJson(response, 200, await getUpdateInfo());
3374:     return true;
3375:   }
3376:
3377:   if (request.method === "GET" && url.pathname === "/api/security-report") {
3378:     if (!isLocalRequest(request)) {
3379:       sendJson(response, 403, { error: "Security reports are only available from this computer." });
3380:       return true;
3381:     }
3382:     sendJson(response, 200, await getSecurityReport());
```

```

3383:     return true;
3384: }
3385:
3386: if (request.method === "GET" && url.pathname === "/api/code/tools") {
3387:   if (!isLocalRequest(request)) {
3388:     sendJson(response, 403, { error: "Compiler details are only available from this computer." });
3389:     return true;
3390:   }
3391:   sendJson(response, 200, { ok: true, tools: await compileToolStatus() });
3392:   return true;
3393: }
3394:
3395: if (request.method !== "POST") return false;
3396:
3397: if (!isLocalRequest(request)) {
3398:   sendJson(response, 403, { error: "Write actions are only allowed from this computer." });
3399:   return true;
3400: }
3401:
3402: if (url.pathname === "/api/portfolio-ai") {
3403:   await handlePortfolioAi(request, response);
3404:   return true;
3405: }
3406:
3407: if (url.pathname === "/api/code/beautify") {
3408:   try {
3409:     const body = await readRequestJson(request);
3410:     const language = normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code,
body.fileName));
3411:     sendJson(response, 200, {
3412:       ok: true,
3413:       language,
3414:       code: beautifyCode(body.code || "", language)
3415:     });
3416:   } catch (error) {
3417:     sendJson(response, 400, { ok: false, error: error.message || "Code could not be beautified." });
3418:   }
3419:   return true;
3420: }
3421:
3422: if (url.pathname === "/api/code/save") {
3423:   try {
3424:     const body = await readRequestJson(request);
3425:     const saved = await saveCompileSource(body);
3426:     sendJson(response, 200, { ok: true, saved });
3427:   } catch (error) {
3428:     sendJson(response, 400, { ok: false, error: error.message || "Code could not be saved." });
3429:   }
3430:   return true;
3431: }
3432:
3433: if (url.pathname === "/api/code/compile") {
3434:   try {
3435:     const body = await readRequestJson(request);
3436:     const result = await compileAndRunCode(body);
3437:     sendJson(response, 200, { ok: result.ok, result });
3438:   } catch (error) {
3439:     sendJson(response, 400, { ok: false, error: error.message || "Code could not be compiled.", result:
null });
3440:   }
3441:   return true;
3442: }
3443:
3444: if (url.pathname === "/api/code/install-tools") {
3445:   try {
3446:     const body = await readRequestJson(request);
3447:     const result = await installCompilerTools(body.language || "");
3448:     sendJson(response, 200, result);
3449:   } catch (error) {
3450:     sendJson(response, 400, { ok: false, error: error.message || "Compiler tools could not be
installed." });
3451:   }
3452:   return true;
3453: }
3454:
3455: if (url.pathname === "/api/publish-target") {
3456:   await readRequestJson(request).catch(() => ({}));
3457:   sendJson(response, 400, {
3458:     ok: false,
3459:     error: "Publishing target setup now requires GitHub authentication.",
3460:     details: "Use Authenticate target. The builder keeps the previous target until GitHub write access
is verified."
3461:   });
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

Representative opening snippet

```

1: import { createServer } from "node:http";
2: import { execFile, spawn } from "node:child_process";
3: import { createHash } from "node:crypto";

```

```

    4: import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from
"node:fs/promises";
    5: import os from "node:os";
    6: import path from "node:path";
    7: import { promisify } from "node:util";
    8: import { fileURLToPath } from "node:url";
    9:
   10: const root = path.dirname(fileURLToPath(import.meta.url));
   11: const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE || root);
   12: const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE || path.join(path.dirname(root),
"compile-code"));
   13: const port = Number(process.env.PORT || 8080);
   14: const host = process.env.HOST || "0.0.0.0";
   15: const execFileAsync = promisify(execFile);
   16: const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() =>
"{}"));
   17:
   18: const types = {
   19:   ".css": "text/css",
   20:   ".csv": "text/csv",
   21:   ".html": "text/html",
   22:   ".ico": "image/x-icon",
   23:   ".js": "application/javascript",
   24:   ".json": "application/json",
   25:   ".md": "text/markdown",
   26:   ".pdf": "application/pdf",
   27:   ".png": "image/png",
   28:   ".jpg": "image/jpeg",
   29:   ".jpeg": "image/jpeg",
   30:   ".svg": "image/svg+xml",
   31:   ".txt": "text/plain",
   32:   ".webp": "image/webp",
   33:   ".xml": "application/xml",
   34:   ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
   35:   ".zip": "application/zip"
   36: };
   37:
   38: const draftPath = path.join(root, "projects.local.json");
   39: const catalogPath = path.join(root, "projects.json");
   40: const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
   41: const publishPaths = [
   42:   "projects.json",
   43:   "docs",
   44:   "assets",
   45:   "index.html",
   46:   "styles.css",
   47:   "script.js",
   48:   "electronics-search.js",
   49:   "Backgrounds",
   50:   ".nojekyll",
   51:   "_headers",
   52:   ".well-known",
   53:   "robots.txt",
   54:   "sitemap.xml",
   55:   "CNAME"
   56: ];
   57: const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
   58: const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
   59: const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
   60: const publishAuthExtendedThreshold = 3;
   61: const defaultSiteRepository = process.env.OMB_BUILDER_REPOSITORY || "";
   62: const blockedAppUpdateVersions = new Set(["0.2.16"]);
   63: const publishAuthorizationHelp = [
   64:   "Publishing was blocked before live website files were applied.",
   65:   "Open Publishing target, enter the repository URL, then click Authenticate target.",
   66:   "A GitHub/Git Credential Manager browser sign-in may open. Sign in with an account that has write
access to the selected Pages repository.",
   67:   "Authenticate target only verifies write access and remembers it for the matching repository and
branch.",
   68:   "Load from target only imports compatible website files from the authenticated repository into the
local builder workspace.",
   69:   "Until a compatible writable website repository is associated, the builder remains local-only."
   70: ].join(" ");
   71: const gitCandidates = [
   72:   process.env.GIT_EXE,

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.